UNIT I PROBLEM SOLVING

Introduction to AI - AI Applications - Problem solving agents – search algorithms – uninformed search strategies – Heuristic search strategies – Local search and optimization problems – adversarial search – constraint satisfaction problems (CSP)

1.Introduction to Artificial Intelligence:

Artificial Intelligence is the branch of computer science concerned with making computers behave like humans. **John McCarthy**, who coined the term in 1956, defines it as "the science and engineering of making intelligent machines, especially intelligent computer programs."
Artificial intelligence is the study of building intelligent machines that can

1. Think Humanly
2. Act Humanly
3. Think Rationally
4. Act Rationally

1.Thinking Humanly: "The cognitive model approach"

If we say the systems have to think humanly then we have to get inside the working of human minds. There are two approaches.

(i) Through Introspection- trying to capture our own thoughts as they go by.

(ii) Through psychological experiments.

Allen Newell and Herbert Simon, who developed **GPS**, the "**General Problem Solver**" tried to trace the reasoning steps to traces of human subjects solving the same problems. The interdisciplinary field of **cognitive science** brings together computer models from AI and experimental techniques from psychology to try to construct precise and testable theories of the workings of the human mind.

Acting Humanly: "The Turing Test approach"

➢ Test proposed by Alan Turing in 1950.

The computer is asked questions by a human interrogator. The computer passes the test if a human interrogator, after posing some written questions, cannot tell whether the written responses come from a person or not.

To pass, the computer needs to possess the following capabilities

- ❖ **Natural language processing** to enable it to communicate successfully in English. ❖ **Knowledge representation** to store what it knows or hears
- ❖ **Automated reasoning** to use the stored information to answer questions and to draw new conclusions.
- ❖ **Machine learning** to adapt to new circumstances and to detect and extrapolate patterns. **The total Turing Test** includes a video signal so that the interrogator can test the subject's perceptual abilities, as well as the opportunity for the interrogator to pass physical objects "through the hatch." To pass the complete Turing Test, the computer will need
  - ❖ **Compute vision** to perceive the objects, and
  - ❖ **Robotics** to manipulate objects and move about.

Thinking rationally: "The laws of thought approach"

- The Greek philosopher Aristotle was one of the first to attempt to codify "right thinking," that is, irrefutable reasoning processes.
- His **syllogisms** provided patterns for argument structures that always yielded correct conclusions when given correct premises-for example, "Socrates is a man; all men are mortal; therefore, Socrates is mortal."
- These laws of thought were supposed to govern the operation of the mind; their study initiated the field called **logic.**

Example: Ram is a student of III year CSE.

- All students are good in III year in CSE.
- Ram is a good student.

Acting Rationally: "The rational agent approach"

An **agent** is something that acts. Computer agents are not mere programs but they are expected to have the following attributes also:
(a) operating under autonomous control,
(b) perceiving their environment,
(c) persisting over a prolonged time period,
(e) adapting to change. A **rational agent** is one that acts so as to achieve the best outcome.

Acting rationally means, to achieve one's goal given one's beliefs. In the previous topic laws of thought approach, correct inference is selected, conclusion is derived, but the agent acts on the conclusion defined the task of acting rationally.
A rational agent is one that acts so as to achieve the best outcome
The study of rational agent has TWO advantages:
i. Correct inference is selected and applied.
ii. It concentrates on scientific development rather than other methods.

Why Artificial Intelligence?

Following are some main reasons to learn about AI:

- With the help of AI, we can create such software or devices which can solve real-world problems very easily and with accuracy such as health issues, marketing, traffic issues, etc.
- With the help of AI, we can create your personal virtual Assistant, such as Cortana, Google Assistant, Siri, etc.
- With the help of AI, we can build such Robots which can work in an environment where survival of humans can be at risk.
- AI opens a path for other new technologies, new devices, and new Opportunities.

Goals of Artificial Intelligence

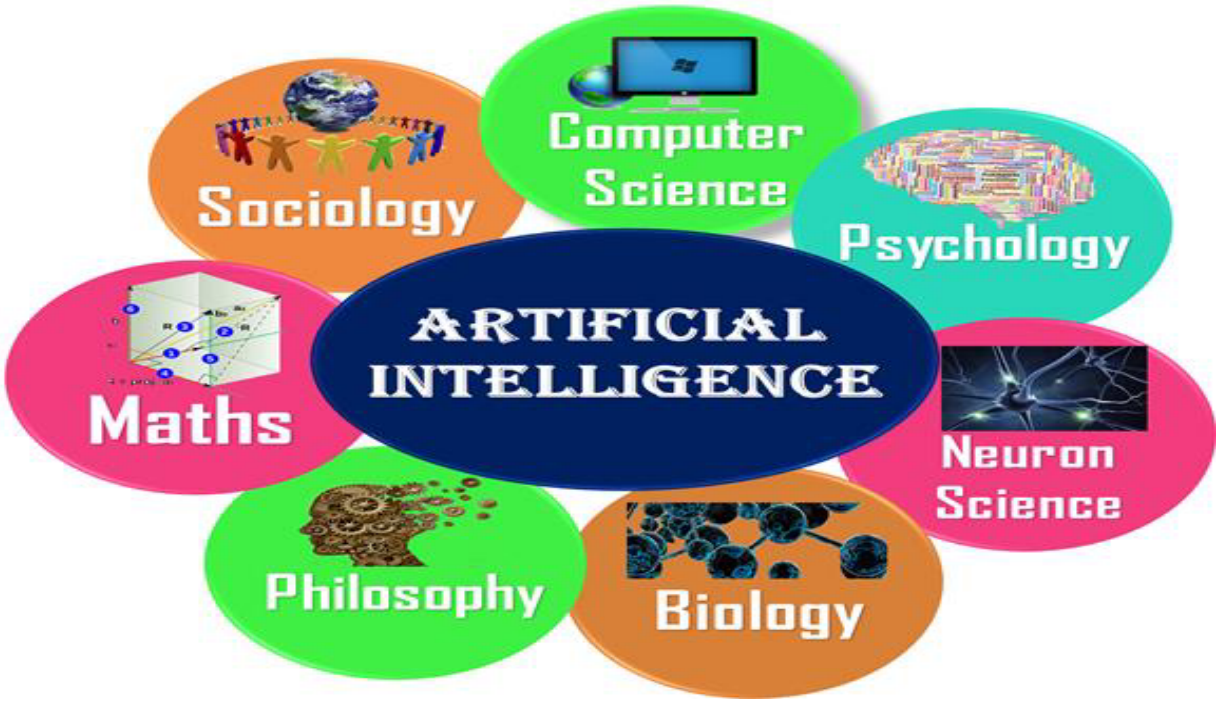
Following are the main goals of Artificial Intelligence:

1. Replicate human intelligence
2. Solve Knowledge-intensive tasks
3. An intelligent connection of perception and action
4. Building a machine which can perform tasks that requires human intelligence such as:
    - Proving a theorem
    - Playing chess
    - Plan some surgical operation
    - Driving a car in traffic
5. Creating some system which can exhibit intelligent behavior, learn new things by itself, demonstrate, explain, and can advise to its user.

What Comprises to Artificial Intelligence?

To create the AI first we should know that how intelligence is composed, so the Intelligence is an intangible part of our brain which is a combination of **Reasoning, learning, problem-solving perception, language understanding, etc**.
To achieve the above factors for a machine or software Artificial Intelligence requires the following discipline:

1. Mathematics
2. Biology
3. Psychology
4. Sociology
5. Computer Science
6. Neurons Study
7. Statistics



<u>Foundations of Artificial Intelligence:</u>

The various disciplines that contributed AI are given below

Philosophy (428 B.C. – present)

Aristotle (384-322 B.C.) was the first to formulate a precise set of laws governing the rational part of the mind. He developed an informal system of syllogisms for proper reasoning, which allowed one to generate conclusions mechanically, given initial premises.

Psychology (1879 – present)

The origin of scientific psychology are traced back to the wok if German physiologist Hermann vonHelmholtz (1821-1894) and his student Wilhelm Wundt(1832 – 1920).In 1879,Wundt opened the first laboratory of experimental psychology at the university of Leipzig. In US, the development of computer modeling led to the creation of the field of **cognitive science**. The field can be said to have started at the workshop in September
1956 at MIT.

Computer engineering (1940-present)

For artificial intelligence to succeed, we need two things: intelligence and an artifact. The computer has been the artifact of choice. **A1** also owes a debt to the software side of computer science, which has supplied the operating systems, programming languages, and tools needed to write modern programs.

Control theory and Cybernetics (1948-present)

Ktesibios of Alexandria (c. 250 B.C) built the first self-controlling machine: a water clock with a regulator that kept the flow of water running through it at a constant, predictable pace. Modern control theory, especially the branch known as stochastic optimal control, has as its goal the design of systems that maximize an **objective function** over time.

Linguistics (1957-present)

Modem linguistics and AI, then, were "born" at about the same time, and grew up together, intersecting in a hybrid field called **computational linguistics** or **natural language processing**

Advantages of Artificial Intelligence

Following are some main advantages of Artificial Intelligence:

- **High Accuracy with less errors:** AI machines or systems are prone to less errors and high accuracy as it takes decisions as per pre-experience or information.
- **High-Speed:** AI systems can be of very high-speed and fast-decision making, because of that AI systems can beat a chess champion in the Chess game.
- **High reliability:** AI machines are highly reliable and can perform the same action multiple times with high accuracy.
- **Useful for risky areas:** AI machines can be helpful in situations such as defusing a bomb, exploring the ocean floor, where to employ a human can be risky.
- **Digital Assistant:** AI can be very useful to provide digital assistant to the users such as AI technology is currently used by various E-commerce websites to show the products as per customer requirement.
- **Useful as a public utility:** AI can be very useful for public utilities such as a self-driving car which can make our journey safer and hassle-free, facial recognition for security purpose, Natural language processing to communicate with the human in human-language, etc.

Disadvantages of Artificial Intelligence

Every technology has some disadvantages, and the same goes for Artificial intelligence. Being so advantageous technology still, it has some disadvantages which we need to keep in our mind while creating an AI system.

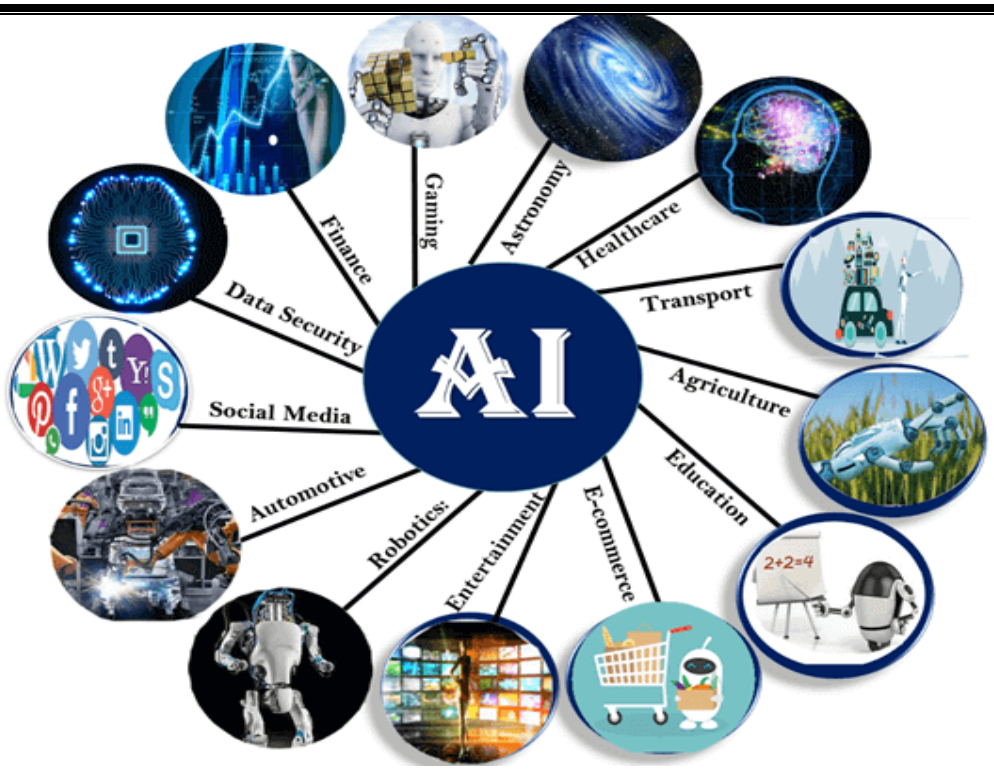
Following are the disadvantages of AI:

- **High Cost:** The hardware and software requirement of AI is very costly as it requires lots of maintenance to meet current world requirements.
- **Can't think out of the box:** Even we are making smarter machines with AI, but still they cannot work out of the box, as the robot will only do that work for which they are trained, or programmed.
- **No feelings and emotions:** AI machines can be an outstanding performer, but still it does not have the feeling so it cannot make any kind of emotional attachment with human, and may sometime be harmful for users if the proper care is not taken.
- **Increase dependency on machines:** With the increment of technology, people are getting more dependent on devices and hence they are losing their mental capabilities.
- **No Original Creativity:** As humans are so creative and can imagine some new ideas but still AI machines cannot beat this power of human intelligence and cannot be creative and imaginative.

2.Applications of AI:

Artificial Intelligence has various applications in today's society. It is becoming essential for today's time because it can solve complex problems with an efficient way in multiple industries, such as Healthcare, entertainment, finance, education, etc. AI is making our daily life more comfortable and fast.

Following are some sectors which have the application of Artificial Intelligence:



1. AI in Astronomy

- Artificial Intelligence can be very useful to solve complex universe problems. AI technology can be helpful for understanding the universe such as how it works, origin, etc.

2. AI in Healthcare

- In the last, five to ten years, AI becoming more advantageous for the healthcare industry and going to have a significant impact on this industry.
- Healthcare Industries are applying AI to make a better and faster diagnosis than humans. AI can help doctors with diagnoses and can inform when patients are worsening so that medical help can reach to the patient before hospitalization.

3. AI in Gaming

- AI can be used for gaming purpose. The AI machines can play strategic games like chess, where the machine needs to think of a large number of possible places.

4. AI in Finance

- AI and finance industries are the best matches for each other. The finance industry is implementing automation, chatbot, adaptive intelligence, algorithm trading, and machine learning into financial processes.

5. AI in Data Security

- The security of data is crucial for every company and cyber-attacks are growing very rapidly in the digital world. AI can be used to make your data more safe and secure. Some examples such as AEG bot, AI2 Platform,are used to determine software bug and cyber-attacks in a better way.

6. AI in Social Media

- Social Media sites such as Facebook, Twitter, and Snapchat contain billions of user profiles, which need to be stored and managed in a very efficient way. AI can organize and manage massive amounts of data. AI can analyze lots of data to identify the latest trends, hashtag, and requirement of different users.

7. AI in Travel & Transport

- AI is becoming highly demanding for travel industries. AI is capable of doing various travel related works such as from making travel arrangement to suggesting the hotels, flights, and best routes to the customers. Travel industries are using AI-powered chatbots which can make human-like interaction with customers for better and fast response.

8. AI in Automotive Industry

- Some Automotive industries are using AI to provide virtual assistant to their user for better performance. Such as Tesla has introduced TeslaBot, an intelligent virtual assistant.

- Various Industries are currently working for developing self-driven cars which can make your journey more safe and secure.

9. AI in Robotics:

- Artificial Intelligence has a remarkable role in Robotics. Usually, general robots are programmed such that they can perform some repetitive task, but with the help of AI, we can create intelligent robots which can perform tasks with their own experiences without pre-programmed.
- Humanoid Robots are best examples for AI in robotics, recently the intelligent Humanoid robot named as Erica and Sophia has been developed which can talk and behave like humans.

10. AI in Entertainment

- We are currently using some AI based applications in our daily life with some entertainment services such as Netflix or Amazon. With the help of ML/AI algorithms, these services show the recommendations for programs or shows.

11. AI in Agriculture

- Agriculture is an area which requires various resources, labor, money, and time for best result. Now a day's agriculture is becoming digital, and AI is emerging in this field. Agriculture is applying AI as agriculture robotics, solid and crop monitoring, predictive analysis. AI in agriculture can be very helpful for farmers.

12. AI in E-commerce

- AI is providing a competitive edge to the e-commerce industry, and it is becoming more demanding in the e-commerce business. AI is helping shoppers to discover associated products with recommended size, color, or even brand.

13. AI in education:

- AI can automate grading so that the tutor can have more time to teach. AI chatbot can communicate with students as a teaching assistant.
- AI in the future can be work as a personal virtual tutor for students, which will be accessible easily at any time and any place.

Examples of AI systems.

Autonomous planning and scheduling: A hundred million miles from Earth, NASA's Remote Agent program became the first on-board autonomous planning program to control the scheduling of operations for a spacecraft (Jonsson *et* al., 2000). Remote Agent generated plans from high-level goals specified from the ground, and it monitored the operation of the spacecraft as the plans were executed-detecting, diagnosing, and recovering from problems as they occurred.

Game playing: IBM's **Deep Blu**e became the first computer program to defeat the world champion in a chess match when it bested Garry Kasparov by a score of 3.5 to 2.5 in an exhibition match (Goodman and Keene, 1997).

Autonomous control: The **ALVINN** computer vision system was trained to steer a car to keep it following a lane. It was placed in CMU's NAVLAB computer-controlled minivan and used to navigate across the United States-for 2850 miles it was in control of steering the vehicle 98% of the time.

Language understanding and problem solving: PROVERB (Littman *et al.,* 1999) is a computer program that solves crossword puzzles better than most humans, using constraints on possible word fillers, a large database of past puzzles, and a variety of information sources including dictionaries and online databases such as a list of movies and the actors that appear in them.

- **Clever Boat**
- IRobot Roomba
- SIRI

History of AI:

Important research that laid the groundwork for AI:

➢ In 1931, Goedel layed the foundation of Theoretical Computer Science**1920-30s**: He published the first universal formal language and showed that math itself is either flawed or allows for unprovable but true statements.

➢ In 1936, Turing reformulated Goedel's result and church's extension thereof. Artificial

Intelligence Page ➢ In 1956, John McCarthy coined the term "Artificial Intelligence" as the topic

of the **Dartmouth Conference**, the first conference devoted to the subject.

➢ In 1957, The **General Problem Solver (GPS)** demonstrated by Newell, Shaw & Simon ➢ In 1958, John McCarthy (MIT) invented the Lisp language.

➢ In 1959, Arthur Samuel (IBM) wrote the first game-playing program, for checkers, to achieve sufficient skill to challenge a world champion.

➢ In 1963, Ivan Sutherland's MIT dissertation on Sketchpad introduced the idea of interactive graphics into computing.

➢ In 1966, Ross Quillian (PhD dissertation, Carnegie Inst. of Technology; now CMU) demonstrated semantic nets

➢ In 1967, Dendral program (Edward Feigenbaum, Joshua Lederberg, Bruce Buchanan, Georgia Sutherland at Stanford) demonstrated to interpret mass spectra on organic chemical compounds. First successful knowledge-based program for scientific reasoning. ➢ In 1967, Doug Engelbart invented the mouse at SRI

➢ In 1968, Marvin Minsky & Seymour Papert publish Perceptrons, demonstrating limits of simple neural nets.

➢ In 1972, Prolog developed by Alain Colmerauer.

➢ In Mid 80's, Neural Networks become widely used with the Backpropagation algorithm (first described by Werbos in 1974).

➢ 1990, Major advances in all areas of AI, with significant demonstrations in machine learning, intelligent tutoring, case-based reasoning, multi-agent planning, scheduling, uncertain reasoning, data mining, natural language understanding and translation, vision, virtual reality, games, and other topics.

➢ In 1997, Deep Blue beats the World Chess Champion Kasparov

➢ In 2002, iRobot, founded by researchers at the MIT Artificial Intelligence Lab, introduced **Roomba**, a vacuum cleaning robot. By 2006, two million had been sold.

3.Problem solving agents

Agents and Environments:

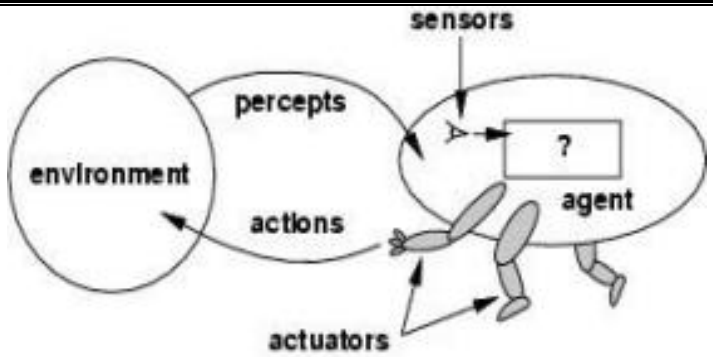
An **agent** is anything that can be viewed as perceiving its environment through sensors and acting upon that environment throuh actuators. An Agent runs in the cycle of **perceiving**, **thinking**, and **acting**.

Different types of agents

An agent can be:

1. **Human-Agent**: A human agent has eyes, ears, and other organs which work for sensors and hand, legs, vocal tract work for actuators.
2. **Robotic Agent**: A robotic agent can have cameras, infrared range finder, NLP for sensors and various motors for actuators.
3. **Software Agent**: Software agent can have keystrokes, file contents as sensory input and act on those inputs and display output on the screen.
4. **Generic agent** - A general structure of an agent who interacts with the environment.

Hence the world around us is full of agents such as thermostat, cellphone, camera, and even we are also agents.



Percept: The term percept is to refer to the agent's perceptual inputs at any given instance. **Action**: It refers to agent's output at any given instance.

Percept Sequence: Agent's percept sequence is the complete history of everything the agent has ever perceived.

Agent Function: An agent's behavior is described by the agent function that maps any given percept sequence to an action.

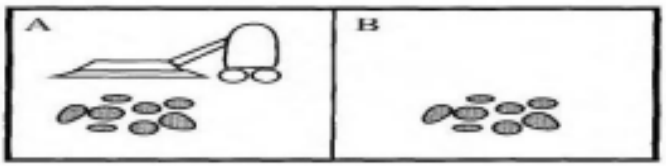
Agent Program: The agent function for an artificial agent will be implemented by an agent program.

Example:

1. The vacuum-cleaner world has just two locations: squares A and B.

2. The vacuum agent perceives which square it is in and whether there is dirt in the square. 3. It can choose to move left, move right, suck up the dirt, or do nothing.

4. Agent function: if the current square is dirty, then suck, otherwise move to the other square.



➢ Percepts: location and status, e.g., [A, Dirty]

➢ Actions: Left, Right, Suck, NoOp

Percept sequence	**Action**
[A, Clean]	Right
[A, Dirty]	Suck
[B, Clean]	Left
[B, Dirty]	Suck

function VACUUM-AGENT([location,status])

```
if status = Dirty then return Suck
else if location = A then return Right
else if location = B then return Left
```

A rational agent:

"For each possible percept sequence, a rational agent should select an action that is expected to maximize its performance measure, given the evidence provided by the percept sequence and whatever built-in knowledge the agent has." A rational agent should be autonomous.

An omniscient agent:

An omniscient agent knows the actual outcome of its actions and can act accordingly; but omniscience is impossible in reality.

Environment: It is defined as an area in which an agent can act.

Agent Environment in AI

Environment: It is defined as an area in which an agent can act.

An environment is everything in the world which surrounds the agent, but it is not a part of an agent itself.

Types of environment:

1. Fully observable vs Partially Observable
2. Static vs Dynamic
3. Discrete vs Continuous
4. Deterministic vs Stochastic
5. Single-agent vs Multi-agent
6. Episodic vs sequential
7. Known vs Unknown
8. Accessible vs Inaccessible

1. Fully observable vs Partially Observable:

1. If an agent sensor can sense or access the complete state of an environment at each point of time then it is **a fully observable** environment.
    a. Eg :Puzzle game
2. If the agent is not able to sense the environment completely then it is called partially observable environment .E.g Card games, Car driving.

- An agent with no sensors in all environments then such an environment is called as unobservable.

2. Deterministic vs Stochastic:

1. If an agent's current state and selected action can completely determine the next state of the environment, then such environment is called a deterministic environment.
2. A stochastic environment is random in nature and cannot be determined completely by an agent.
3. In a deterministic, fully observable environment, agent does not need to worry about uncertainty.

Examples:

Chess – there would be only a few possible moves for a coin at the current state and these moves can be determined.

Self-Driving Cars- the actions of a self-driving car are not unique, it varies time to time.

3. Episodic vs Sequential:

1. If an episodic environment, the agent's experience is divided into episodes.
2. Each episode consists of its own projects and actions and it does not depend on the previous episode. In an episodic environment, there is a series of one-shot actions, and only the current percept is required for the action.
3. In Sequential environment, an agent requires memory of past actions to determine the next best actions.

Real-life Example: Playing tennis is a perfect example where a player observes the opponent's shot and takes action.

4.Discrete vs Continuous: If the number of distinct percepts and actions is limited, the environment is discrete, otherwise it is continuous.

Example: Taxi driving is a continuous state and continuous-time problem and A self-driving car is an example of a continuous environment.

- If in an environment there are a finite number of percepts and actions that can be performed within it, then such an environment is called a discrete environment else it is called continuous environment.

  Example: A chess gamecomes under discrete environment as there is a finite number of moves that can be performed.

5. Single-agent vs Multi-agent

1. If only one agent is involved in an environment, and operating by itself then such an environment is called single agent environment.
2. If multiple agents are operating in an environment, then such an environment is called a multi-agent environment.
3. The agent design problems in the multi-agent environment are different from single agent environment.

   Example: An agent solving a crossword puzzle by itself is clearly in a single-agent environment, An agent playing chess is in a two-agent environment.

6. Static vs Dynamic:

1. If the environment can change itself while an agent is deliberating, then such environment is called a dynamic environment else it is called a static environment.
2. Static environments are easy to deal because an agent does not need to continue looking at the world while deciding for an action.
3. However for dynamic environment, agents need to keep looking at the world at each action.
4. Taxi driving is an example of a dynamic environment whereas Crossword puzzles are an example of a static environment.

7. Known vs Unknown

1. In a known environment, the results for all actions are known to the agent. While in unknown environment, agent needs to learn how it works in order to perform an action.
2. It is quite possible that a known environment to be partially observable and an Unknown environment to be fully observable.

8. Accessible vs Inaccessible

1. If an agent can obtain complete and accurate information about the state's environment, then such an environment is called an Accessible environment else it is called inaccessible.
2. An empty room whose state can be defined by its temperature is an example of an accessible environment.
3. Information about an event on earth is an example of Inaccessible environment.

The following Table lists the properties of a number of familiar environments:

Task Environment	Observable	Deterministic	Episodic	Static	Discrete	Agents
Crossword puzzle	Fully	Deterministic	Sequential	Static	Discrete	Single
Chess with a clock	Fully	Strategic	Sequential	Semi	Discrete	Multi
Poker	Partially	Strategic	Sequential	Static	Discrete	Multi
Backgammon	Fully	Stochastic	Sequential	Static	Discrete	Multi
Taxi driving	Partially	Stochastic	Sequential	Dynamic	Continuous	Multi
Medical diagnosis	Partially	Stochastic	Sequential	Dynamic	Continuous	Single

Image analysis	Fully	Deterministic	Episodic	Semi	Continuous	Single
Part-picking robot	Partially	Stochastic	Episodic	Dynamic	Continuous	Single
Refinery controller	Partially	Stochastic	Sequential	Dynamic	Continuous	Single
Interactive English tutor	Partially	Stochastic	Sequential	Dynamic	Discrete	Multi

Example: PEAS description of the task environment for agents

Agent Type	**Performance Measure**	**Environment**	**Actuators**	**Sensors**
Automated Taxi Driver	Safe, fast, legal, comfortable trip, maximize profits	Roads, traffic, pedestrian customers	Steering accelerator, brake, signal, horn,	Cameras, sonar, speedometer, GPS, odometer, accelerometer engine sensors,
Medical diagnosis system	Healthy patient, minimize costs, lawsuits	Patient, hospital, staff	Screen display (question tests, diagnoses treatment	Keyboard (entry of symptoms, findings, patient's answers)
Part-Picking Robot	Percentage of parts in correct bin	Conveyor belt with parts, bins	Jointed arm and hand	Camera, joint angle sensors
Interactive English tutor	Maximize student's score on test	Set of students	Screen display (exercises)	Keyboard
robot soccer player	amount of goals scored	soccer match field	legs	cameras, sonar or infrared
Satellite Image Analysis	Correct Image Categorization	Downlink from satellite	Display categorization of scene	Colour pixel arrays
Refinery controller	Maximum purity, safety, yield	Refinery operators	Valves, pumps, heaters, displays	Temperature, pressure, chemical sensors

Vacuum Agent	minimize energy consumption, maximize dirt	two squares	Left, Right, Suck, NoOp	Sensors to identify the dirt

Problem solving agent is the goal based agent, where the agent decides what to do by finding sequence of actions that lead to desirable states. The agent can adopt a goal and aim at satisfying it

Problem: Any task given to an agent is called as a problem. A problem is really a collection of information
that the agent will use to decide what to do.
The solution of many problems can be described by finding a sequence of actions that lead to a desirable
goal. Each action changes the state and the aim is to find the sequence of actions and states that lead from
the initial (start) state to a final (goal) state.
Problem formulation is the process of deciding what actions and states to consider given a goal.

Types:

Incremental Formulation:

- Start from the initial state
- Generate sequence of actions
- Explore until goal state is reached.

Complete state Formulation:

- Start with the basic state.
- Check for the conditions and generate sequence of actions.
- Continue until goal is reached.

A problem can be defined formally by **four components**:

1. **Initial state** –The state from which the agent starts.
2. **Successor function**- It is a function which provides sequence of actions. For a particular state x successor fn(x) returns a set of ordered pairs (action, successor), Where each action is a legal action and successor is a state that can be reached from x by applying the action.
3. **Goal test -** Determines whether the given state is the goal state.
4. **Path cost -** A function that assigns numerical cost to each path.

A solution to a problem is a path from the initial state to a goal state

Operator - The set of possible actions available to the agent.

State space (or) state set space - The set of all possible states reachable from the initial state by any sequence of actions.

Path (state space) - The sequence of action leading from one state to another.

The effectiveness of a search can be measured using three factors. They are:

1 Solution is identified or not?

2. Is it a good solution? If yes, then path cost to be minimum.

3. Search cost of the problem that is associated with time and memory required to find a solution.

Examples:

i) Vacuum world Problem

States: The agent is in one of two locations, each of which might or might not contain dirt. Thus there are 2 * 2^2 = 8 possible world states.

Initial state: Any state can be designated as the initial state.

Successor function: three actions (Left, *Right,* and *Suck).*

Goal test: This checks whether all the squares are clean.

Path cost: Each step costs 1, so the path cost is the number of steps in the path.

The 8 possible states of the simplified vacuum world problem.

ii)8-Square Problem:

1. The 8-square problem consists of a 3 x 3 board with eight numbered tiles and a blank space.
2. A tile adjacent to the blank space can slide into the space.
3. The object is to reach a specified goal state

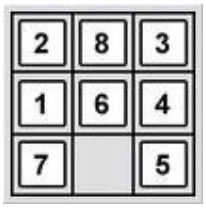
States: A state description specifies the location of each of the eight tiles and the blank in one of the nine squares.

Initial state: Any state can be designated as the initial state.

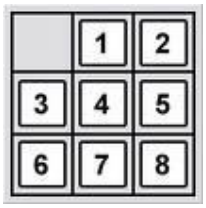
Successor function: This generates the legal states that result from trying the four actions (blank moves Left, Right, Up, or Down).

Goal test: This checks whether the state matches the goal configuration (Other goal configurations are possible.)

Path cost: Each step costs 1, so the path cost is the number of steps in the path.



Initial State



Goal State

iii)Water Jug Problem

1. Given two jugs, a 4-gallon one and a 3-gallon one.
2. Neither have any measuring markers on it.
3. There is a pump that can be used to fill the jugs with water.
4. How can you get exactly 2 gallons of water into the 4-gallon jug?

States: The state space for this problem can be described as the set of ordered pairs of integers(x,y) such that x=0,1,2,3, or 4 and y=0,1,2, or 3:x represents the number of gallons of water in the 4-gallon jug, and y represents the quantity of the water in the 3-gallon jug.

Initial state: The starting state is (0, 0) where 3 –gallon jug and 4- gallon jug are empty

Successor function: This generates the legal productions that result from trying the 12 actions (empty the jug, Pour some water from jug, Fill the jug).

1. (x, y) if x<4	**-> (4 , y)**
2. (x, y) if y<3	**->(x , 3)**
3. (x, y) if x>0	**->(x-d , y)**
4. (x, y) if y>0	**->(x ,y-d)**
5. (x, y) if x>0	**->(0 , y)**
6. (x, y) if y>0	**->(x, 0)**
7. (x, y) if x+y>=4 and y>0	**-> (4 , y-(4-x))**
8. (x, y) if x+y>=3 and x>0	**->(x-(3-y), 3)**
9. (x, y) if x+y<=4 and y>0	**->(x+y, 0)**
10. (x, y) if x+y<=3 and x>0	**-> (0, x+y)**
11. (0, 2)	**-> (2, 0)**
12. (2, y)	**-> (0, y)**

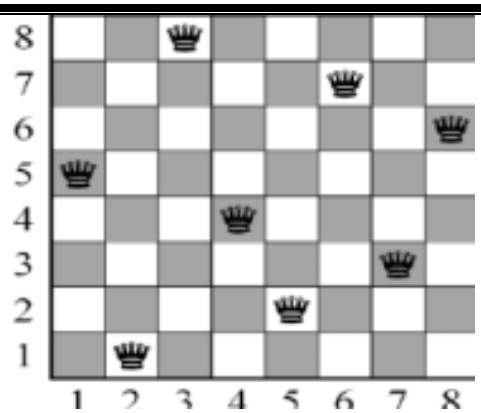
Goal test: The goal state is (2, n) for any value of n (the problem does not specify how many gallons need to be in the 3-gallon jug)

Path cost: Each step costs 1, so the path cost is the number of steps in the path.

Gallons in the 4-Gallon Jug	**Gallon in the 3-Gallon Jug**	**Rule Applied**
0	0	2
0	3	9
3	0	2
3	3	7
4	2	5 or 12
0	2	9 or 11
2	0	

iv)**8-queens problem:** The aim of this problem is to place eight queens on a chessboard in an order where no queen may attack another. A queen can attack other queens either **diagonally or in same row and column.**

From the following figure, we can understand the problem as well as its correct solution.



It is noticed from the above figure that each queen is set into the chessboard in a position where no other queen is placed diagonally, in same row or column. Therefore, it is one right approach to the 8-queens problem.

Problem Solving by Search

An important aspect of intelligence is ***goal-based*** problem solving.

The **solution** of many **problems can** be described by finding a **sequence of actions** that lead to a desirable **goal.** Each action changes the ***state*** and the aim is to find the sequence of actions and states that lead from the initial (start) state to a final (goal) state.

A well-defined problem can be described by:

- Initial state
- **Operator or successor function** - for any state x returns s(x), the set of states reachable from x with one action
- **State space** - all states reachable from initial by any sequence of actions
- **Path** - sequence through state space
- **Path cost** - function that assigns a cost to a path. Cost of a path is the sum of costs of individual actions along the path
- **Goal test** - test to determine if at goal state

4.Search Techniques:

What is Search?

Search is the systematic examination of **states** to find path from the **start/root state** to the **goal state.** The set of possible states, together with *operators* defining their connectivity constitute the *search space*.

The output of a search algorithm is a solution, that is, a path from the initial state to a state that satisfies the goal test.

Search Tree

Having formulated some **problems**, we now need to **solve** them. This is done by a **search** through the **state space**. A **search tree** is generated by the **initial state** and the **successor function** that together define the **state space**. In general, we may have a *search graph* rather than a *search tree*, when the same state can be reached from multiple paths.

The choice of which state to expand is determined by the **search strategy**. There are various search strategies.

Search Algorithm Terminologies:

1. **Search:** Searching is a step by step procedure to solve a search-problem in a given search space. A search problem can have three main factors:
   - **Search Space:** Search space represents a set of possible solutions, which a system may have.
   - **Start State:** It is a state from where agent begins **the search**.
   - **Goal test:** It is a function which observe the current state and returns whether the goal state is achieved or not.
2. **Search tree:** A tree representation of search problem is called Search tree. The root of the search tree is the root node which is corresponding to the initial state.
3. **Actions:** It gives the description of all the available actions to the agent.
4. **Transition model:** A description of what each action do, can be represented as a transition model.
5. **Path Cost:** It is a function which assigns a numeric cost to each path.
6. **Solution:** It is an action sequence which leads from the start node to the goal node.
7. **Optimal Solution:** If a solution has the lowest cost among all solutions.

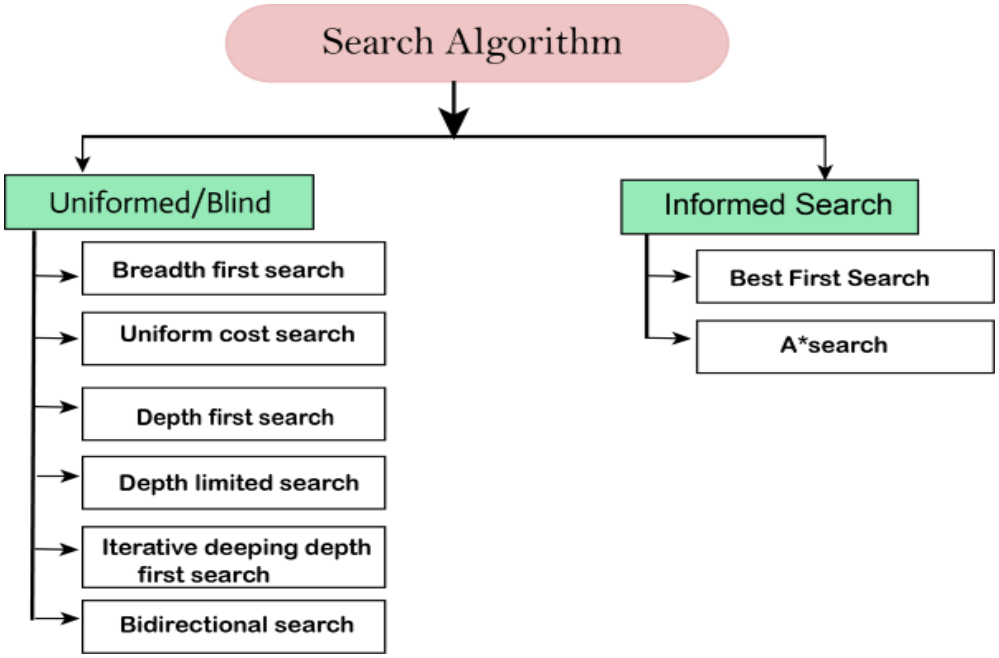
Properties of Search Algorithms:

Following are the four essential properties of search algorithms to compare the efficiency of these algorithms:

1. **Completeness:** A search algorithm is said to be complete if it guarantees to return a solution if at least any solution exists for any random input.
2. **Optimality:** If a solution found for an algorithm is guaranteed to be the best solution (lowest path cost) among all other solutions, then such a solution for is said to be an optimal solution.
3. **Time Complexity:** Time complexity is a measure of time for an algorithm to complete its task.
4. **Space Complexity:** It is the maximum storage space required at any point during the search, as the complexity of the problem.

Types of search algorithms

Based on the search problems we can classify the search algorithms into uninformed (Blind search) search and informed search (Heuristic search) algorithms.



Uninformed/Blind Search: Uninformed Search Strategies have no additional information about states beyond that provided in the problem definition. It is also known as Blind Search.

1. The uninformed search does not contain any domain knowledge such as closeness, the location of the goal.
2. It operates in a brute-force way as it only includes information about how to traverse the tree and how to identify leaf and goal nodes.
3. Uninformed search applies a way in which search tree is searched without any information about the search space like initial state operators and test for the goal, so it is also called blind search.
4. It examines each node of the tree until it achieves the goal node.

It can be divided into five main types:

1. Breadth-first search
2. Uniform cost search
3. Depth-first search
4. Iterative deepening depth-first search
5. Bidirectional Search

Informed Search

Informed search algorithms use domain knowledge. In an informed search, problem information is available which can guide the search. Informed search strategies can find a solution more efficiently than an uninformed search strategy. Informed search is also called a Heuristic search.

A heuristic is a way which might not always be guaranteed for best solutions but guaranteed to find a good solution in reasonable time.

Informed search can solve much complex problem which could not be solved in another way.

An example of informed search algorithms is a traveling salesman problem.

1. Greedy Search

2. A* Search

5.Uninformed Search Strategies: Uninformed search is a class of general-purpose search algorithms which operates in brute force-way. Uninformed search algorithms do not have additional information about state or search space other than how to traverse the tree, so it is also called blind search.

Uninformed Search Strategies have no additional information about states beyond that provided in the problem definition. It is also known as Blind Search.

Various uninformed search strategies are:

1. Breadth First Search
2. Depth first search
3. Depth limited search
4 .Iterative deepening search
5. Uniform cost search
6. Bidirectional search

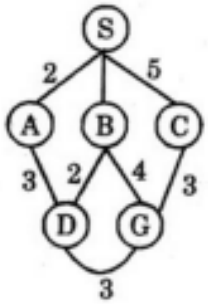
Breadth First Search:

1. Breadth-first search is the most common search strategy for traversing a tree or graph. This algorithm searches breadthwise in a tree or graph, so it is called breadth-first search.
2. BFS algorithm starts searching from the root node of the tree and expands all successor node at the current level before moving to nodes of next level.
3. The breadth-first search algorithm is an example of a general-graph search algorithm.
4. Breadth-first search implemented using FIFO queue data structure.

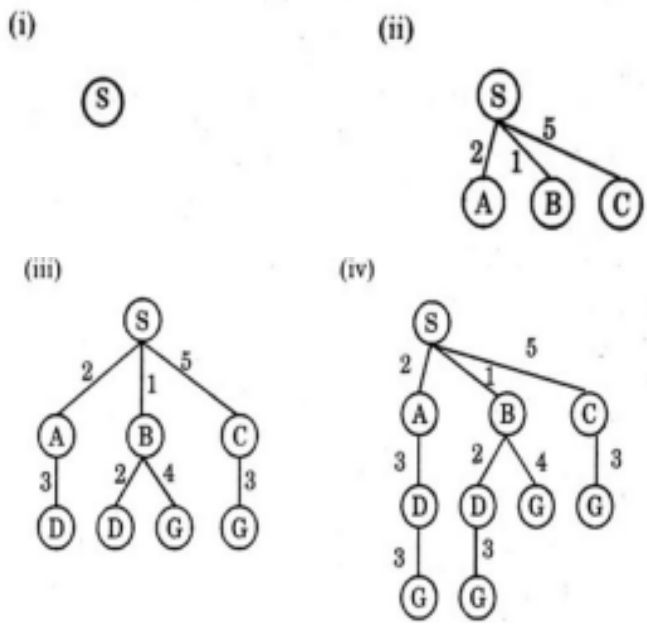
. ➢ In other words, calling TREE-SEARCH(Problem, FIFO-QUEUE())results in a breadth-first search.

➢ The FIFO queue puts all newly generated successors at the end of the queue, which means that shallow nodes are expanded before deeper nodes.

Example 1:



Task: Find a ,path from. S to G using BFS



The path in the 2nd depth level is selected, (i.e) SBG{or) SCG.

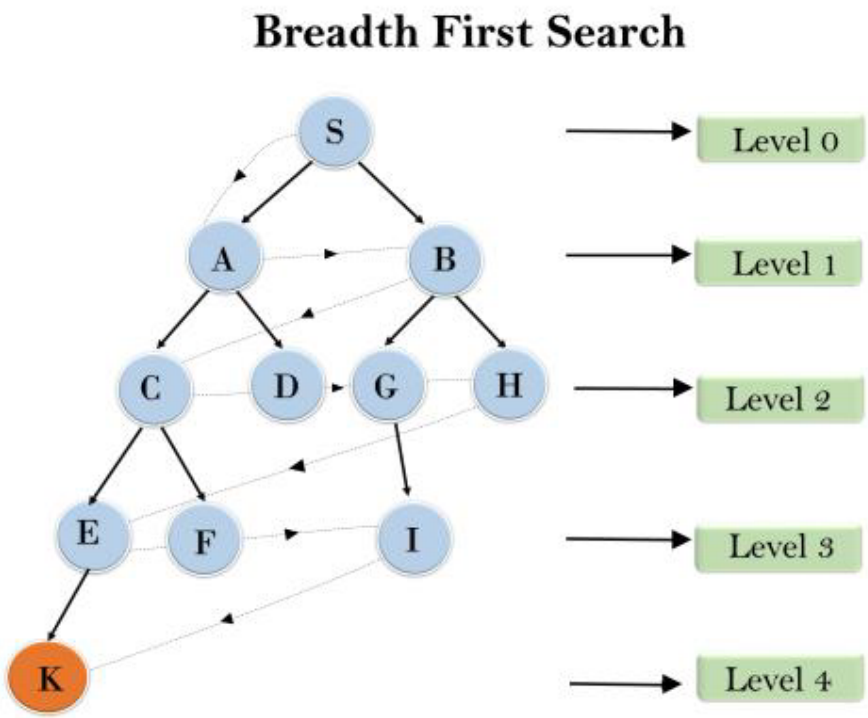
Algorithm :

function BFS{problem) returns a solution or failure

return TREE-SEARCH (problem, FIFO-QUEUE( ))

Example2:

In the below tree structure, we have shown the traversing of the tree using BFS algorithm from the root node S to goal node K. BFS search algorithm traverse in layers, so it will follow the path which is shown by the dotted arrow, and the traversed path will be:



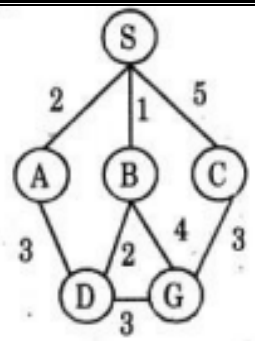
Performance Measures:

- Completeness - Yes
- Optimality- Yes
- Time complexity- O(b^d)
- Space complexity- O(b^d)

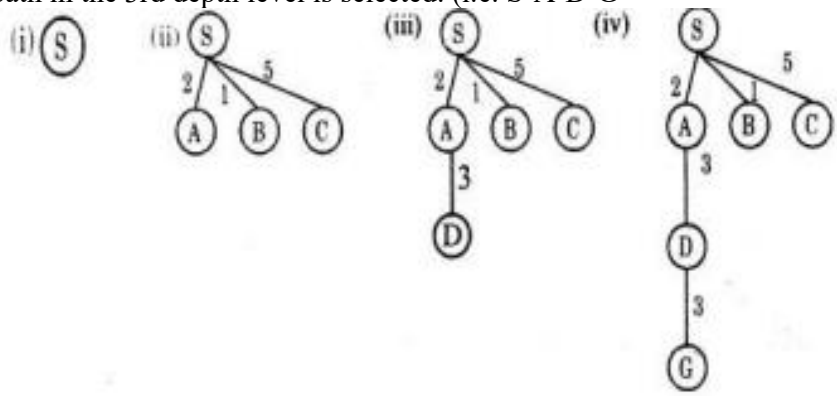
Depth-First-Search

1. Depth-first-search always expands the deepest node in the current fringe of the search tree.
2. The progress of the search is illustrated in figure .
3. The search proceeds immediately to the deepest level of the search tree, where the nodes have no successors.
4. As those nodes are expanded, they are dropped from the fringe, so then the search "backs up" to the next shallowest node that still has unexplored successors.
5. This strategy can be implemented by TREE-SEARCH with a last-in-first-out (LIFO) queue, also known as a stack.
6. Depth-first-search has very modest memory requirements.
7. It needs to store only a single path from the root to a leaf node, along with the remaining unexpanded sibling nodes for each node on the path.
8. Once the node has been expanded, it can be removed from the memory, as soon as its descendants have been fully explored. For a state space with a branching factor b and maximum depth m,depth-first-search requires storage of only bm + 1 nodes.

Example1: **Route finding problem**



The path in the 3rd depth level is selected. (i.e. S-A-D-G



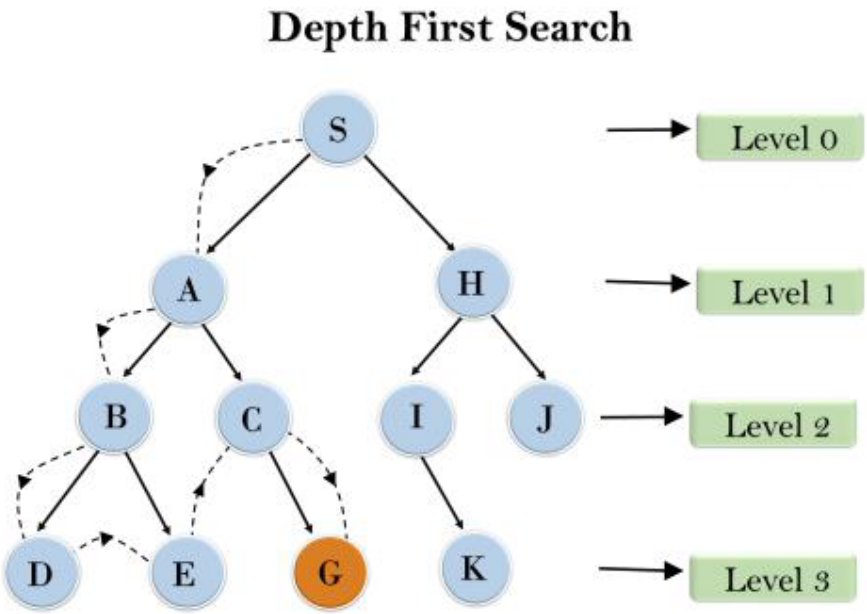
Algorithm:

function *DFS(problem)* **return** a solution or failure
TREE-SEARCH(problem, LIFO-QUEUE())

Example 2: In the below search tree, we have shown the flow of depth-first search, and it will follow the order as:

- Root node--->Left node ----> right node.

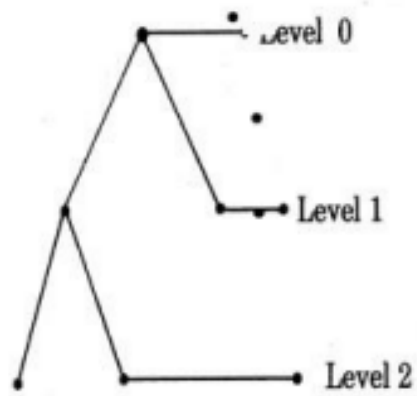
1. It will start searching from root node S, and traverse A, then B, then D and E, after traversing E, it will backtrack the tree as E has no other successor and still goal node is not found.
2. After backtracking it will traverse node C and then G, and here it will terminate as it found goal node.



Performance Measures:

Time and space complexity:
In the worst case depth first search has to expand all the nodes.
Time complexity : $O(b^m)$.
The nodes are expanded towards one particular direction requires memory

for only that nodes. **Space complexity** : $O(b^m)$.



b=2
m = 2 :. bm=4

Completeness: No

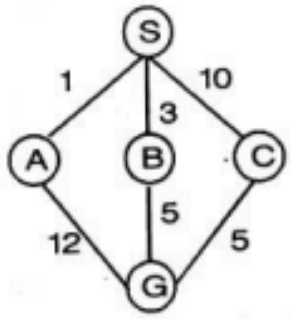
Optimality: No

Advantage: If more than one solution exists (or) number of levels is high then DFS is best because exploration is done only in a small portion of the whole space.
Disadvantage: Not guaranteed to find a solution.

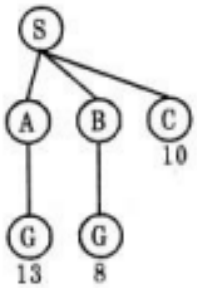
Uniform-cost search

1. Breadth-first search is optimal when all step costs are equal, because it always expands the shallowest unexpanded node.
2. By a simple extension, we can find an algorithm that is optimal with any step cost function.
3. Instead of expanding the shallowest node, uniform-cost search expands the node n with the lowest path cost.
4. The root node is expanded first, then the next node to be expanded is selected as the lower cost node on the fringe rather than the lowest depth node. ie., g(n)=path cost.
5. Breadth first search is equivalent to uniform cost search when g(n)=DEPTH(n).
6. Note that if all step costs are equal, this is identical to breadth-first search.
7. Uniform-cost search does not care about the number of steps a path has, but only about their total cost.

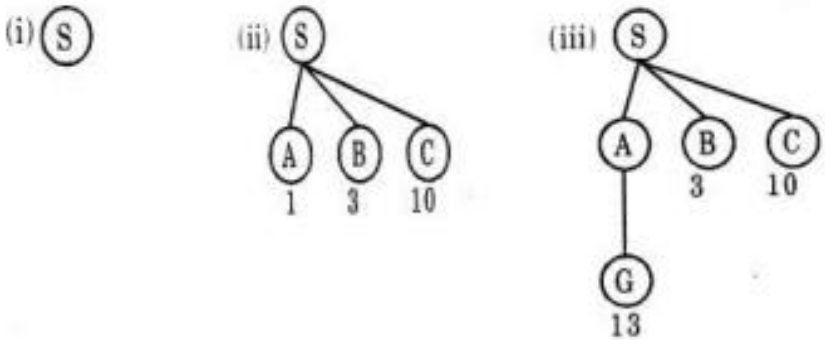
Example: **Route finding problem**



Task **: Find a minimum path cost from S to G**

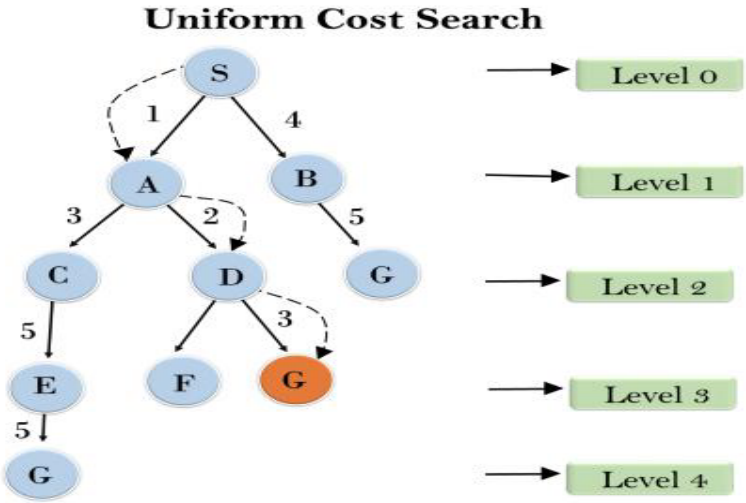


Since the value of A is less it is expanded first, but it is not optimal.
B to be expanded next



SBG is the path with minimum path cost. No need to expand the next path SC, because its path cost is high to reach C from S, as well as goal state is reached in the previous path with minimum cost.

Example 2:



Performance Measures:

:Time complexity is same as breadth first search because instead of depth level the minimum path cost is considered.

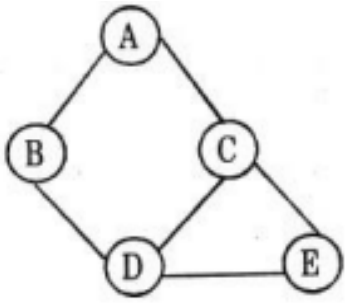
- Time complexity: $O(b^d)$
- Space complexity: $O(b^d)$
- Completeness: Yes
- Optimality: Yes

Advantage: Guaranteed to find the single solution at minimum path cost.
Disadvantage: Suitable for only smallest instances problem.

Depth - limited search

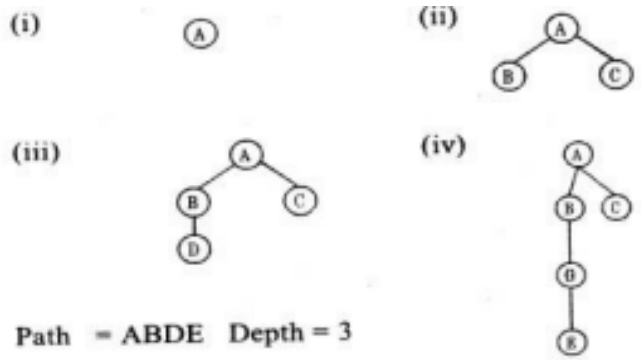
1. *Definition:* A cut off (maximum level of the depth) is introduced in this search technique to overcome the disadvantage of depth first search. The cutoff value depends on the number of states.

Example: Route finding problem



The number of states in the given map is 5. So, it is possible to get the goal state at a maximum depth of 4. Therefore the cutoff value is 4

***Task* : Find a path from A to E.**



Performance Measures:

The worst case time complexity is equivalent to BFS and worst case DFS.

Time complexity : O(bl). The nodes which is expanded in one particular direction above to be stored.

Space complexity: O(bl)

Optimality: No, because not guaranteed to find the shortest solution first in the search technique.

Completeness: Yes, guaranteed to find the solution if it exists.

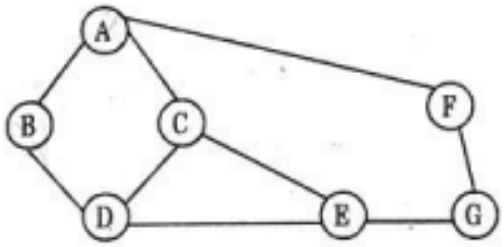
Advantage: Cut off level is introduced in the DFS technique

Disadvantage: Not guaranteed to find the optimal solution.

Iterative deepening search **Iterative deepening search**

Definition: Iterative deepening search is a strategy that sidesteps the issue of choosing the best depth limit by trying all possible depth limits.

Example: Route finding problem



Task: Find a path from A to G

Limit 0:

Limit:1

Limit 2:

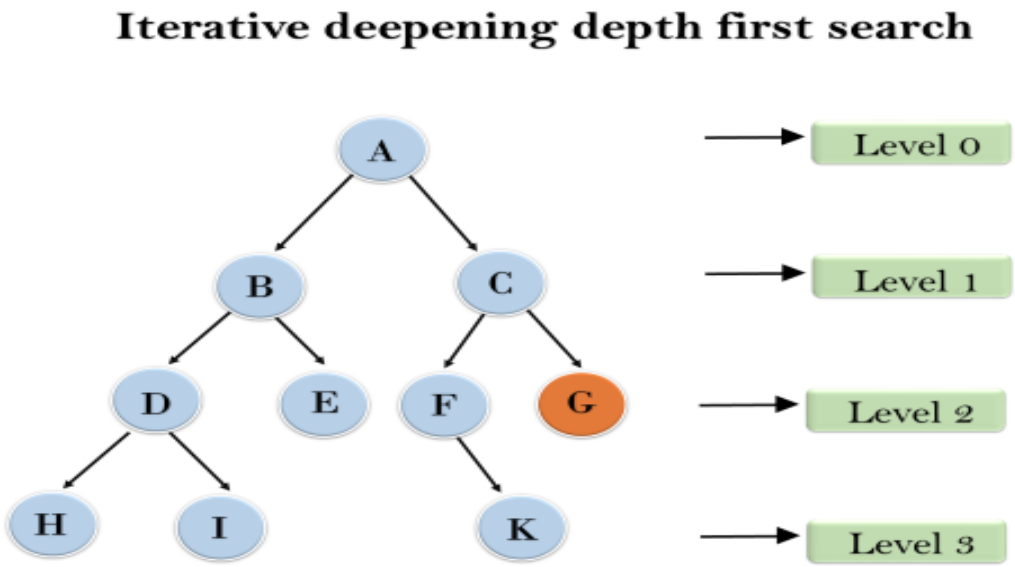
Solution: The goal state G can be reached from A in four ways. They are:

1. A - B - D - E - G ------- Limit 4
2. A - B - D - E - G ------- Limit 4
3. A - C - E - G ------- Limit 3
4. A - F - G ------ Limit2

Since it is a iterative deepening search it selects lowest depth limit (i.e.) A-F-G is selected as the solution path.

Example 2:

Following tree structure is showing the iterative deepening depth-first search. IDDFS algorithm performs various iterations until it does not find the goal node. The iteration performed by the algorithm is given as:



1'st Iteration-----> A

2'nd Iteration----> A, B, C

3'rd Iteration------>A, B, D, E, C, F, G

4'th Iteration------>A, B, D, H, I, E, C, F, K, G

Performance Measures:

Iterative deepening combines the advantage of breadth first search and depth first search (i.e) expansion of states is done as BFS and memory requirement is equivalent to DFS.

Time Complexity : O(bd)

Space Complexity :O(bd)

Optimality: Yes, because the order of expansion of states is similar to breadth first search.

Completeness: yes, guaranteed to find the solution if it exists.

Advantage: This method is preferred for large state space and the depth of the search is not known.

Disadvantage: Many states are expanded multiple times. Example: The state D is expanded twice in limit

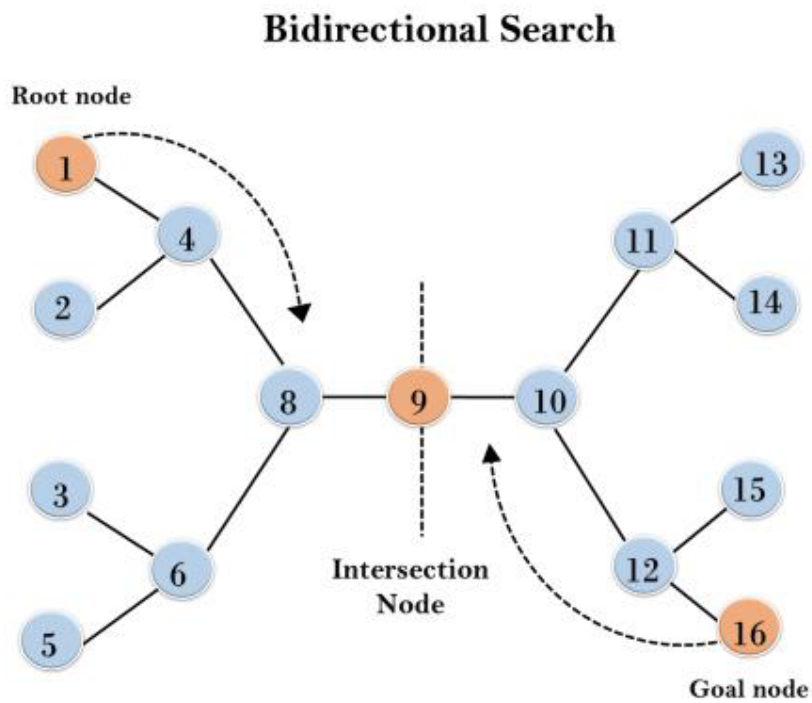
2. <u>Bidirectional search</u>

Definition : Bidirectional search is a strategy that simultaneously search both the directions (i.e.) forward from the initial state and backward from the goal, and stops when the two searches meet in the middle. Example: Route finding problem

Task : Find a path from A to E.

Search from forward (A) :

Search from backward (E) :



Performance Measures:

The forward and backward searches done at the same time will lead to the solution in $O(2b^{d/2}) = O(b^{d/2})$ step, because search is done to go only halfway If the two searches meet at all, the nodes of at least one of them must all be retained in memory requires $O(b^{d/2})$ space.

Optimality: Yes, because the order of expansion of states is done in both the directions.

Completeness: Yes, guaranteed to find the solution if it exists.

Advantage: Time and space complexity is reduced.

Disadvantage:

1. If two searches (forward, backward) does not meet at all, complexity arises in the search technique.
2. In backward search calculating predecessor is difficult task.
3. If more than one goal state 'exists then explicit, multiple state search is required.

Comparing uninformed search strategies

Algorithm	Time	Space	Complete	Optimal
Breadth First	$O(b^d)$	$O(b^d)$	Yes	Yes
Uniform Cost Search	$O(b^{1+floor(C^*/\epsilon)})$	$O(b^{1+floor(C^*/\epsilon)})$	Yes	Yes
Depth First	$O(b^m)$	$O(bm)$	No	No
Depth Limited	$O(b^l)$	$O(bl)$	No	No
Iterative Deepening	$O(b^d)$	$O(bd)$	Yes	Yes
Bidirectional	$O(b^{d/2})$	$O(b^{d/2})$	Yes	Yes

6.Informed (Heuristic) Search Algorithms

Informed search strategy is one that uses problem-specific knowledge beyond the definition of the problem itself. It can find solutions more efficiently than uninformed strategy.

- Informed search algorithm contains an array of knowledge such as how far we are from the goal, path cost, how to reach to goal node, etc.
- This knowledge help agents to explore less to the search space and find more efficiently the goal node.
- The informed search algorithm is more useful for large search space. Informed search algorithm uses the idea of heuristic, so it is also called Heuristic search.

Heuristics function:

1. A **heuristic function** or simply a **heuristic** is a function that ranks alternatives in various search algorithms at each branching step basing on an available information in order to make a decision which branch is to be followed during a search.

2. The key component of Best-first search algorithm is a **heuristic function**, denoted by

h(n): h(n) = estimated cost of the **cheapest path** from node n to a **goal node**.

- Heuristic is a function which is used in Informed Search, and it finds the most promising path.
- It takes the current state of the agent as its input and produces the estimation of how close agent is from the goal.
- The heuristic method, however, might not always give the best solution, but it guaranteed to find a good solution in reasonable time.
- Heuristic function estimates how close a state is to the goal.
- It is represented by h(n), and it calculates the cost of an optimal path between the pair of states.
- The value of the heuristic function is always positive.

Heuristic function is given as:

$h(n) <= h^*(n)$

Here h(n) is heuristic cost, and h*(n) is the estimated cost. Hence heuristic cost should be less than or equal to the estimated cost.

Pure Heuristic Search:

1. Pure heuristic search is the simplest form of heuristic search algorithms.
2. It expands nodes based on their heuristic value h(n).
3. It maintains two lists, OPEN and CLOSED list.
4. In the CLOSED list, it places those nodes which have already expanded and in the OPEN list, it places nodes which have yet not been expanded.
5. On each iteration, each node n with the lowest heuristic value is expanded and generates all its successors and n is placed to the closed list.

The algorithm continues unit a goal state is found. In the informed search we will discuss two main algorithms which are given below:

1. **Best First Search Algorithm(Greedy search)**
2. **A* Search Algorithm**

1.Greedy Best First Search :

One of the simplest best-first search strategies is to minimize the estimated cost to reach the goal. That is, the node whose state is judged to be closest to the goal state is always expanded first. For most problems, the cost of reaching the goal from a particular state can be estimated but cannot be determined exactly.

A function that calculates such cost estimates is called a heuristic function, and is usually denoted by the letter h:

h(n) = estimated cost of the cheapest path from the state at node n to a goal state.

A best-first search that uses h to select the next node to expand is called greedy search, for reasons that will become clear.

1.) Best-first Search Algorithm (Greedy Search):

1. Greedy best-first search algorithm always selects the path which appears best at that moment.
2. It is the combination of depth-first search and breadth-first search algorithms.
3. It uses the heuristic function and search.
4. Best-first search allows us to take the advantages of both algorithms.
5. With the help of best-first search, at each step, we can choose the most promising node.
6. In the best first search algorithm, we expand the node which is closest to the goal node and the closest cost is estimated by heuristic function, i.e.

$$f(n)= g(n).$$

Were, h(n)= estimated cost from node n to the goal.

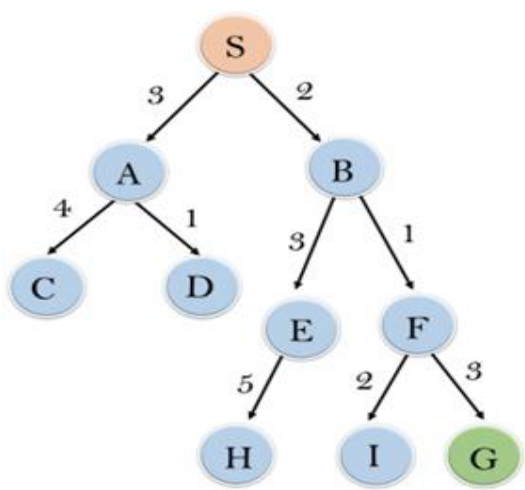
The greedy best first algorithm is implemented by the priority queue.

Best first search algorithm:

Step 1: Place the starting node into the OPEN list.
Step 2: If the OPEN list is empty, Stop and return failure.
Step 3: Remove the node n, from the OPEN list which has the lowest value of h(n), and places it in the CLOSED list.
Step 4: Expand the node n, and generate the successors of node n.
Step 5: Check each successor of node n, and find whether any node is a goal node or not. If any successor node is goal node, then return success and terminate the search, else proceed to Step 6.
Step 6: For each successor node, algorithm checks for evaluation function f(n), and then check if the node has been in either OPEN or CLOSED list. If the node has not been in both list, then add it to the OPEN list.
Step 7: Return to Step 2.

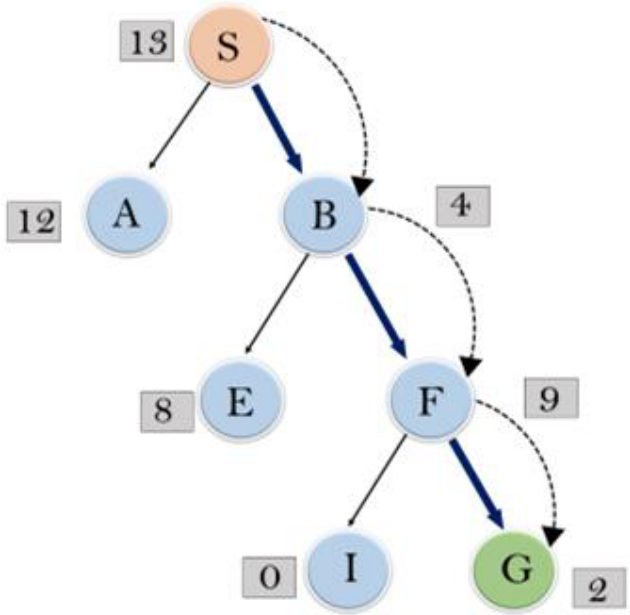
Example:

Consider the below search problem, and we will traverse it using greedy best-first search. At each iteration, each node is expanded using evaluation function f(n)=h(n) , which is given in the below table.



node	H (n)
A	12
B	4
C	7
D	3
E	8
F	2
H	4
I	9
S	13
G	0

In this search example, we are using two lists which are **OPEN** and **CLOSED** Lists. Following are the iteration for traversing the above example.



Expand the nodes of S and put in the CLOSED list

Initialization: Open [A, B], Closed [S]

Iteration 1: Open [A], Closed [S, B]

Iteration2: Open[E,F,A],Closed[S,B]

: Open [E, A], Closed [S, B, F]

Iteration3: Open[I,G,E,A],Closed[S,B,F]

: Open [I, E, A], Closed [S, B, F, G]

Hence the final solution path will be: **S----> B----->F----> G**

Time Complexity: The worst case time complexity of Greedy best first search is $O(b^m)$.

Space Complexity: The worst case space complexity of Greedy best first search is $O(b^m)$. Where, m is the maximum depth of the search space.

Complete: Greedy best-first search is also incomplete, even if the given state space is finite.

Optimal: Greedy best first search algorithm is not optimal.

Advantages:

- Best first search can switch between BFS and DFS by gaining the advantages of both the algorithms.
- This algorithm is more efficient than BFS and DFS algorithms.

Disadvantages:

- It can behave as an unguided depth-first search in the worst case scenario.
- It can get stuck in a loop as DFS.
- This algorithm is not optimal.

Example:

Consider the below search problem, and we will traverse it using greedy best-first search. At each iteration, each node is expanded using evaluation function f(n)=h(n) , which is given in the below table.

2.A* Search :

Greedy search minimizes the estimated cost to the goal, h(n), and thereby cuts the search cost considerably. Unfortunately, it is neither optimal nor complete. Uniform-cost search, on the other hand, minimizes the cost of the path so far, g(ri); it is optimal and complete, but can be very inefficient. It would be nice if we could combine these two strategies to get the advantages of both.

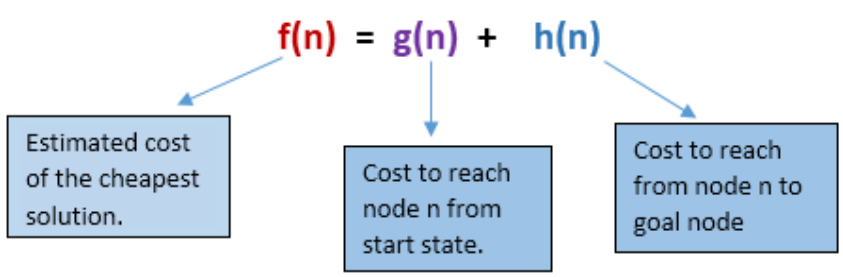
Fortunately, we can do exactly that, combining the two evaluation functions simply by summing them: f(n) = g(n) + h(n).

Since g(n) gives the path cost from the start node to node n, and h(ri) is the estimated cost of the cheapest path from n to the goal, we have /(«) = estimated cost of the cheapest solution through n.

A* Search Algorithm:

A* search is the most commonly known form of best-first search. It uses heuristic function h(n), and cost to reach the node n from the start state g(n). It has combined features of UCS and greedy best-first search, by which it solve the problem efficiently. A* search algorithm finds the shortest path through the search space using the heuristic function. This search algorithm expands less search tree and provides optimal result faster. A* algorithm is similar to UCS except that it uses g(n)+h(n) instead of g(n).

In A* search algorithm, we use search heuristic as well as the cost to reach the node. Hence we can combine both costs as following, and this sum is called as a **fitness number**.



At each point in the search space, only those node is expanded which have the lowest value of f(n), and the algorithm terminates when the goal node is found.

Algorithm of A* search:

Step1: Place the starting node in the OPEN list.

Step 2: Check if the OPEN list is empty or not, if the list is empty then return failure and stops.

Step 3: Select the node from the OPEN list which has the smallest value of evaluation function (g+h), if node n is goal node then return success and stop, otherwise

Step 4: Expand node n and generate all of its successors, and put n into the closed list. For each successor n', check whether n' is already in the OPEN or CLOSED list, if not then compute evaluation function for n' and place into Open list.

Step 5: Else if node n' is already in OPEN and CLOSED, then it should be attached to the back pointer which reflects the lowest g(n') value.

Step 6: Return to **Step 2**.

Advantages:

- A* search algorithm is the best algorithm than other search algorithms.
- A* search algorithm is optimal and complete.
- This algorithm can solve very complex problems.

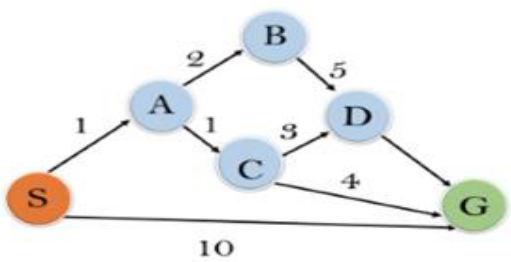
Disadvantages:

- It does not always produce the shortest path as it mostly based on heuristics and approximation.
- A* search algorithm has some complexity issues.
- The main drawback of A* is memory requirement as it keeps all generated nodes in the memory, so it is not practical for various large-scale problems.

Example:

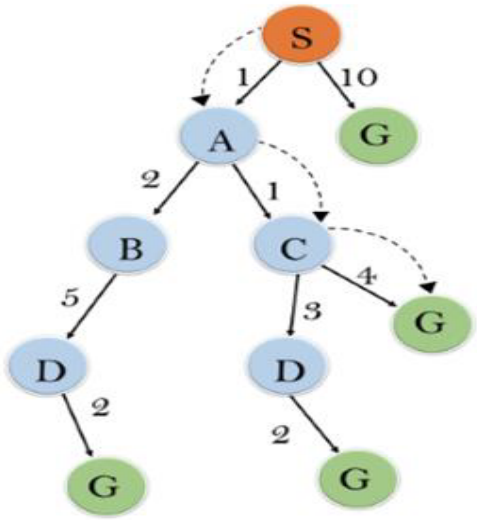
In this example, we will traverse the given graph using the A* algorithm. The heuristic value of all states is given in the below table so we will calculate the f(n) of each state using the formula f(n)= g(n) + h(n), where g(n) is the cost to reach any node from start state.

Here we will use OPEN and CLOSED list.



State	h(n)
S	5
A	3
B	4
C	2
D	6
G	0

Solution:



Initialization: {(S, 5)}

Iteration1: {(S--> A, 4), (S-->G, 10)}

Iteration2: {(S--> A-->C, 4), (S--> A-->B, 7), (S-->G, 10)}

Iteration3: {(S--> A-->C--->G, 6), (S--> A-->C--->D, 11), (S--> A-->B, 7), (S-->G, 10)}

Iteration 4 will give the final result, as **S--->A--->C--->G** it provides the optimal path with cost 6.

Points to remember:

- A* algorithm returns the path which occurred first, and it does not search for all remaining paths.
- The efficiency of A* algorithm depends on the quality of heuristic.
- A* algorithm expands all nodes which satisfy the condition f(n)<="" li="">

Complete: A* algorithm is complete as long as:

- Branching factor is finite.
- Cost at every action is fixed.

Optimal: A* search algorithm is optimal if it follows below two conditions:

- **Admissible:** the first condition requires for optimality is that h(n) should be an admissible heuristic for A* tree search. An admissible heuristic is optimistic in nature.
- **Consistency:** Second required condition is consistency for only A* graph-search.

If the heuristic function is admissible, then A* tree search will always find the least cost path.

Time Complexity: The time complexity of A* search algorithm depends on heuristic function, and the number of nodes expanded is exponential to the depth of solution d. So the time complexity is O(b^d), where b is the branching factor.

Space Complexity: The space complexity of A* search algorithm is **O(b^d)**

<u>Memory Bounded Heuristic Search :</u>

SMA* (**Simplified Memory Bounded A***) is a shortest path algorithm that is based on the A* algorithm. The difference between SMA* and A* is that SMA* uses a bounded memory, while the A* algorithm might need exponential memory.

Like the A*, it expands the most promising branches according to the heuristic. What sets SMA* apart is that it prunes nodes whose expansion has revealed less promising than expected.

SMA*, just like A* evaluates nodes by combining g(n), the cost to reach the node, and h(n), the cost to get from the node to the goal: f(n) = g(n) + h(n).

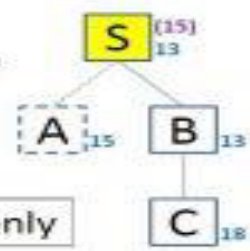
Since g(n) gives the path cost from the start node to node n, and h(n) is the estimated cost of the cheapest path from n to the goal, we have f(n) = estimated cost of the cheapest solution through n. The lower the f value is, the higher priority the node will have.

The difference from A* is that the f value of the parent node will be updated to reflect changes to this estimate when its children are expanded. A fully expanded node will have an f value at least as high as that of its successors.

In addition, the node stores the f value of the best forgotten successor (or best forgotten child). This value is restored if the forgotten successor is revealed to be the most promising successor.

SMA* Algorithm

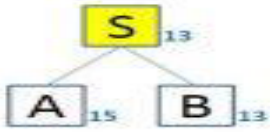
- Optimizes A* to work within reduced memory
- ***Key Idea:***
  - **IF** memory **full** for **extra node (C)**
  - **Remove highest f-value leaf (A)**
  - **Remember best-forgotten child** in *each* parent node **(15 in S)**



E.g. Memory of 3 nodes only

SMA* Algorithm

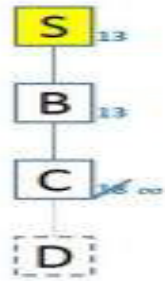
- **Generate Children 1 by 1**
  - **Expanding**: add 1 child at the time to QUEUE
  - Avoids **memory overflow**
  - **Allows monitoring** if nodes need deletion



First add A later B

SMA* Algorithm

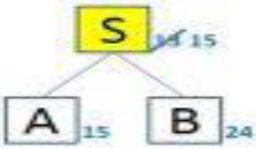
- **Too long paths: Give up**
  - **Extending** path **cannot fit** in **memory**
    - **give up (C)**
  - Set **f-value** node **(C)** to ∞
    - **Remembers:** path cannot be found here



E.g. Memory of 3 nodes only

SMA* Algorithm

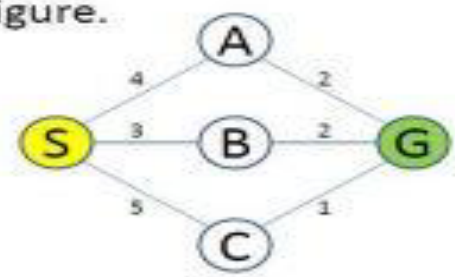
- **Adjust f-values**
  - **IF** all children M_i of node N have been explored
  - **AND** $\forall i$: $f(S...M_i) > f(S...N)$
  - **THEN reset** (through N $\Longrightarrow$ through children)
    - $f(S...N) = \min\{f(S...M_i) \mid M_i \text{ child of } N\}$



Better estimate for f(S)

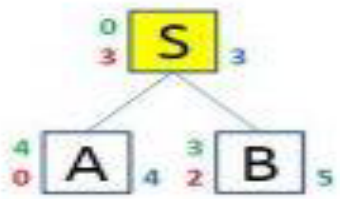
SMA* by Example

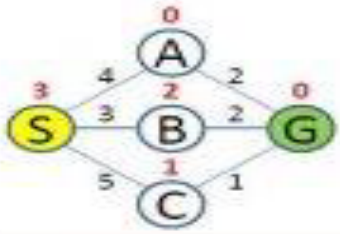
- Perform SMA* (memory: 3 nodes) on the following figure.



	S	A	B	C	G
heuristic	3	0	2	1	0

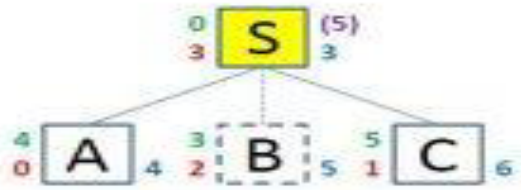
SMA* by Example

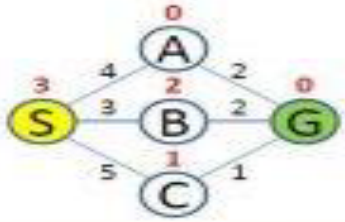




Generate children
(One by one)

SMA* by Example

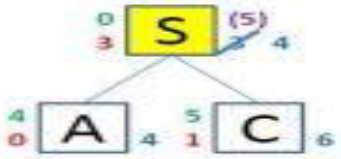


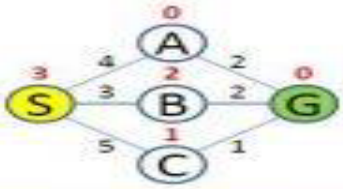


Generate children
(One by one)

Memory full

SMA* by Example





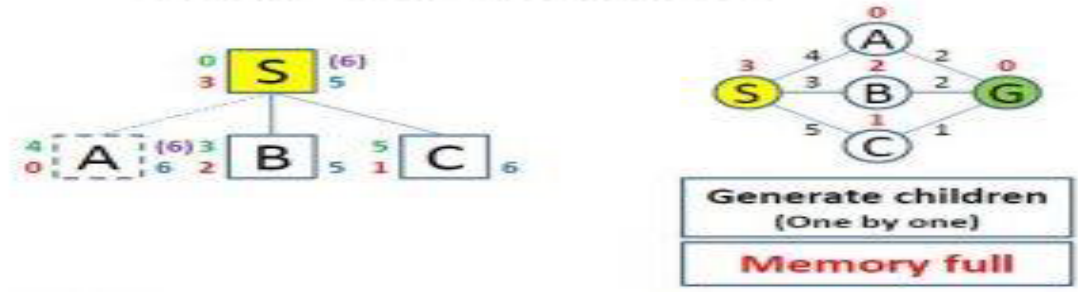
All children are
explored

Adjust f-values

SMA* by Example

SMA* by Example

SMA* by Example



SMA* by Example

(i) If h(n) is consistent then the values of f(n) along any path are non decreasing.

(ii) n' is a successor of n, then

g(n') = g(n) + C (n,a,n') for some a.

Therefore, f(n') = g(n') + h(n')

= g(n) + C(n, a, n') + h(n')

≥ g(n) +h(n) = f(n)

(iii) The sequence of nodes expanded by A* using Graph search is non decreasing order of f(n). The nodes are selected in the same way to reach the goal node. If consistency is maintained at each successor level then optimality is achieved.

3. Completeness:

A* search expands nodes in order of increasing f(i.e) f(n) < f*. This condition may fails when infinite number of nodes are exists with f(n) < f*. How the search technique generates infinite number of nodes? They are:

(i) There is a node with an infinite branching factor.

(ii) There is a path with a finite path coat but an infinite number of nodes along it. Thus the A* is complete on locally finite graphs (graphs with a finite branching factor) provided there is some constant d such that every operator costs at least d.

Memory Bounded Heuristic Search

- **Iterative Deepening A*(IDA*)**

The main difference between IDA* and standard iterative deepening is that the cutoff used is f-cost(g+h) rather than the depth.

- **Recursive Best First Search:**

- It is a simple recursive algorithm that attempts to mimic the operation of standard best first search, but using linear space.
- It uses the *f-limit* variable to keep track of the *f-value* of the best alternative path available from any ancestor of the current.

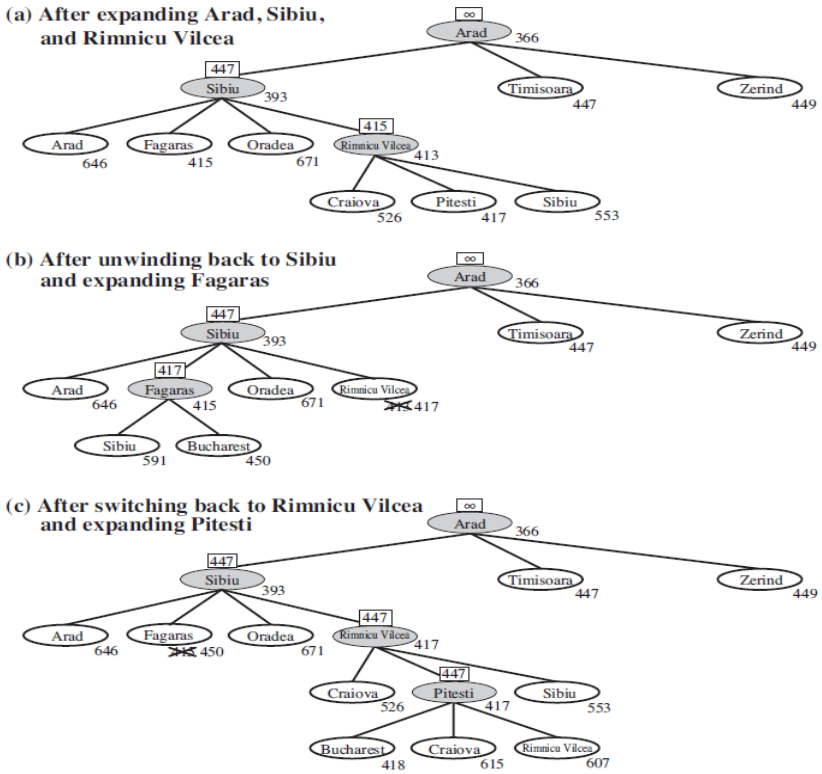
Recursive Best First Search (RBFS)

- If the current node exceeds the limit, the recursion unwinds back to the alternative path.
- As the recursion unwinds, RBFS replaces the *f-value* of each node along the path with a backed up value the best *f-value* of its children.
- Like A* search, RBFS is an optimal algorithm if the heuristic function h(n) is admissible.
- IDA* and RBFS suffer from using too little memory. Even if more memory were available, RBFS has no way to make use of it. Because they forget most of what they have done.(Expanding same states many times over)

```
function RECURSIVE-BEST-FIRST-SEARCH(problem) returns a solution, or failure
  return RBFS(problem, MAKE-NODE(problem.INITIAL-STATE), ∞)

function RBFS(problem, node, f_limit) returns a solution, or failure and a new f-cost limit
  if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)
  successors ← [ ]
  for each action in problem.ACTIONS(node.STATE) do
      add CHILD-NODE(problem, node, action) into successors
  if successors is empty then return failure, ∞
  for each s in successors do /* update f with value from previous search, if any */
      s.f ← max(s.g + s.h, node.f))
  loop do
      best ← the lowest f-value node in successors
      if best.f > f_limit then return failure, best.f
      alternative ← the second-lowest f-value among successors
      result, best.f ← RBFS(problem, best, min(f_limit, alternative))
      if result ≠ failure then return result
```

Figure 3.26 The algorithm for recursive best-first search.



- MA* is to use all available memory.
- SMA*(Simplified MA*) expanding best leaf until memory is full.
- If the memory is full to add a new node SMA* always drops the *worst leaf node* –the one with highest value.
- If all the leaf nodes have same *f-value*, SMA* expands the *newest* best leaf and deletes the *oldest* worst leaf.
- Thrashing-repeated regenerations of the same nodes.

Local Search Algorithms And Optimization Problems:

In many optimization problems, the path to the goal is irrelevant; the goal state itself is the solution.

The best state is identified from the objective function or heuristic cost function. In such cases, we can use local search algorithms (ie) keep only a single current state, try to improve it instead of the whole search space explored so far.

For example, in the 8-queens problem, what matters is the final configuration of queens, not the order in which they are added.

Local search algorithms operate a single current state (rather than multiple paths) and generally move only to neighbours of that state. Typically, the paths followed by the search are not retained.

They have two key advantages:

(1) They use very little memory-usually a constant amount; (2) They can often find reasonable solutions in large or infinite (continuous) state spaces for which systematic algorithms are unsuitable.

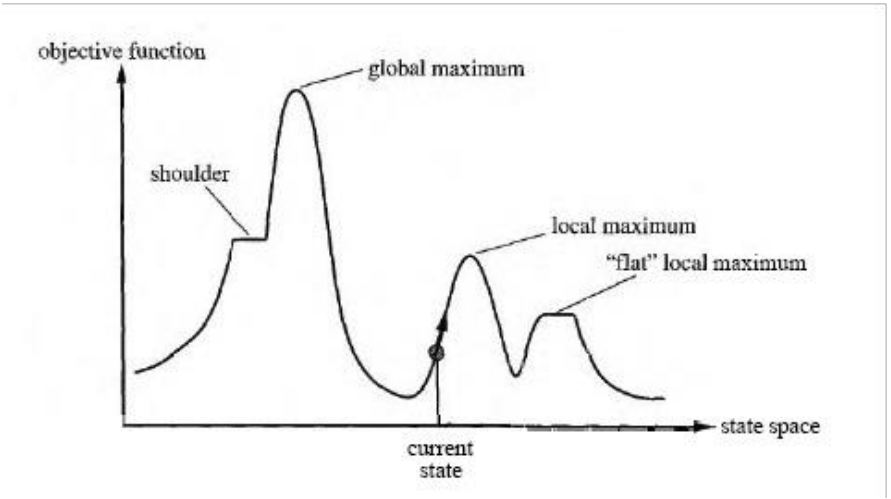
The local search problem is explained with the state space land scape. A landscape has:

Location - defined by the state

Elevation - defined by the value of the heuristic cost function or objective function, if elevation corresponds to cost then the lowest valley (global minimum) is achieved. If elevation corresponds to an objective function, then the highest peak (global maximum) is achieved.

A complete local search algorithm always finds a goal if one exists, an optimal algorithm always finds a global minimum/maximum.

A one-dimensional state space landscape in which elevation corresponds to the objective function.



Applications

Integrated - circuit design
Factory - floor layout
Job-shop scheduling
Automatic programming
Vehicle routing
Telecommunications network Optimization

Advantages

1. Constant search space. It is suitable for online and offline search
2. The search cost is less when compare to informed search.
3. Reasonable solutions are derived in large or continuous state space for which systematic algorithms are unsuitable.

Some of the local search algorithms are:

1. Hill climbing search (Greedy Local Search)
2. Simulated annealing
3. Local beam search
4. Genetic Algorithm (GA)

HILL CLIMBING SEARCH (GREEDY LOCAL SEARCH)

1. The hill-climbing search algorithm is simply a loop that continually moves in the direction of increasing value.
2. It terminates when it reaches a "peak" where no neighbor has a higher value.
3. The algorithm does not maintain a search tree, so the current node data structure need only record the state and its objective function value.
4. At each step the current node is replaced by the best neighbor;

Hill-climbing search algorithm

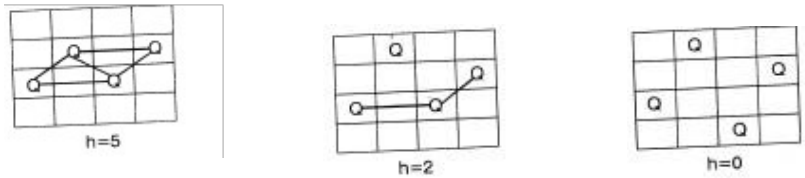
function HILL-CLIMBING(problem) **returns** a state that is a local maximum
inputs: *problem*, a problem
local variables:*current*, a node and *neighbor*, a node
current <- MAKE-NODE(INITIAL-STATE[*problem*])
loop do
neighbor <- a highest-valued successor of current
if VALUE[neighbor] <= VALUE[current] **then return** STATE[*current*]
current <- *neighbor*

To illustrate hill-climbing, we will use the 8-queens, where each state has 8 queens on the board, one per column. The successor function returns all possible states generated by moving a single queen to another square in the same column (so each state has 8 x 7 = 56 successors).

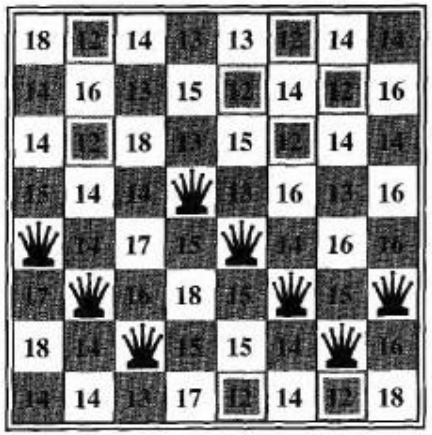
Hill-climbing algorithms typically choose randomly among the set of best successors, if there is more than one.

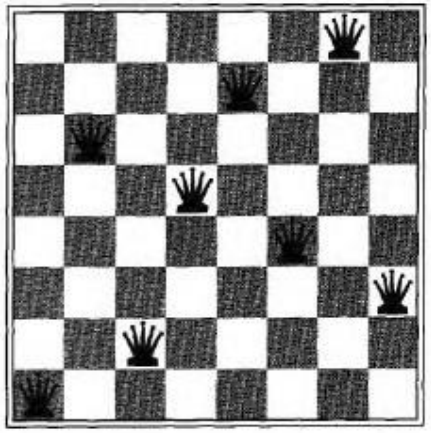
The heuristic cost function h is the number of pairs of queens that are attacking each other, either directly or indirectly.

The global minimum of this function is zero, which occurs only at perfect solutions.



An 8-queens state with heuristic cost estimate h = 17, showing the value of h for each possible successor obtained by moving a queen within its column. The best moves are marked.





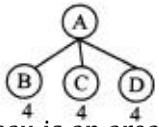
A local minimum in the 8-queens state space; the state has h = 1 but every successor has a higher cost.

Hill climbing often gets stuck for the following reasons:

Local maxima or foot hills : a local maximum is a peak that is higher than each of its neighboring states, but lower than the global maximum

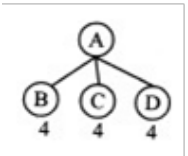
Example :

The evaluation function value is maximum at C and from their there is no path exist for expansion. Therefore C is called as local maxima. To avoid this state, random node is selected using back tracking to the previous node.



Plateau or shoulder: a plateau is an area of the state space landscape where the evaluation function is flat. It can be a flat local maximum.

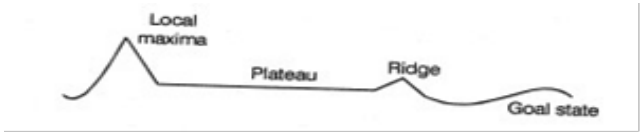
Example :



The evaluation function value of B C D are same, this is a state space of plateau. To avoid these state, random nodes is selected or skip the level (i.e) select the node in the next level

Ridges: Ridges result in a sequence of local maxima that is very difficult for greedy algorithms to navigate. But the disadvantage is more calculations to be done that correspond to moving in several directions at once.

Structure of hill climbing drawbacks



Variants of hill-climbing

Stochastic hill climbing - Stochastic hill climbing chooses at random from among the uphill moves; the probability of selection can vary with the steepness of the uphill move.

First-choice hill climbing - First-choice hill climbing implements stochastic hill climbing by generating successors randomly until one is generated that is better than the current state

Random-restart hill climbing - Random-restart hill climbing adopts the well-known adage, "If at first you don't succeed, try, try again." It conducts a series of hill-climbing searches from randomly generated initial state, stopping when a goal is found.

ADVERSARIAL SEARCH (GAME SEARCH)

Optimal Decision in Games

A game can be formally defined as a kind of search problem with the following components:

1. The initial state, which includes the board position and identifies the player to move.
2. A successor function, which returns a list of (move, state) pairs, each indicating a legal move and the resulting state.
3. A terminal test, which determines when the galme is over. States where the game has ended are called terminal states.
4. A utility function (also called an objective function or payoff function), which gives a numeric value for the terminal states. In chess, the outcome is a win, loss, or draw, with values +1, -1, or 0.
5. The initial state and the legal moves for each side define the game tree for the game.
6. Figure 1 shows part of the game tree for tic-tac-toe (noughts and crosses).
7. From the initial state, MAX has nine possible moves.
8. Play alternates between MAX'S placing an X and MIN'S placing an O until we reach leaf nodes corresponding to terminal states such that one player has three in a row or all the squares are filled.
9. The number on each leaf node indicates the utility value of the terminal state from the point of view of MAX; high values are assumed to be good for MAX and bad for MIN (which is how the players get their names).
10. It is MAX'S job to use the search tree (particularly the utility of terminal states) to determine the best move.

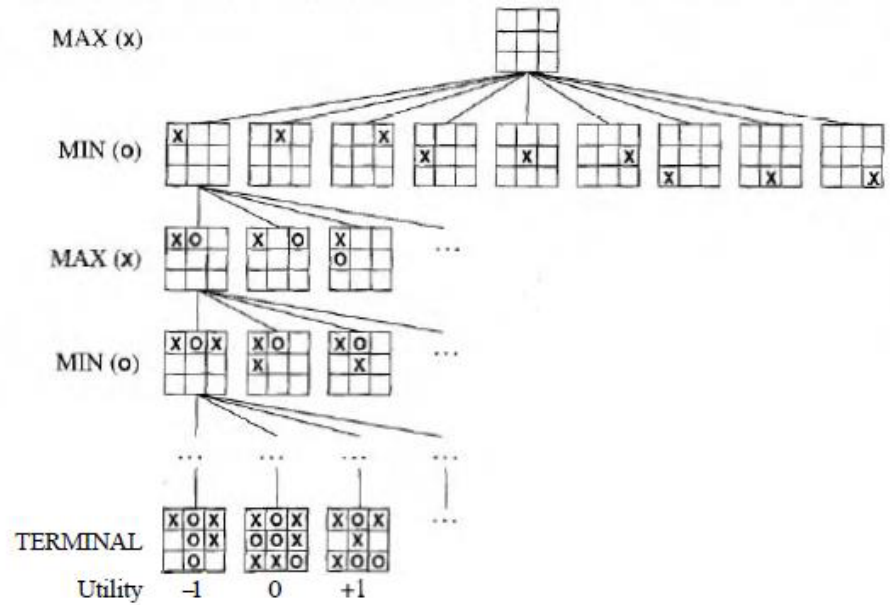


Figure 1: A (partial) search tree for the game of tic-tac-toe. The top node is the initial state, and MAX moves first, placing an X in an empty square. We show part of the search tree, giving alternating moves by MIN (0) and MAX, until we eventually reach terminal states, which can be assigned utilities according to the rules of the game.

Optimal strategies

1. In a normal search problem, the optimal solution would be a sequence of moves leading to a goal state-a terminal state that is a win.
2. The possible moves for MAX at the root node are labelled a1, a2, and a3.
3. The possible replies to a1 for MIN are b1, b2, b3, and so on.

4. This particular game ends after one move each by MAX and MIN.
5. (In game parlance, we say that this tree is one move deep, consisting of two half-moves, each of which is called a ply.) The utilities of the terminal states in this game range from 2 to 14.
6. Given a game tree, the optimal strategy can be determined by examining the minimax value of each node, which we write as MINIMAX-VALUE (n).
7. The minimax value of a node is the utility (for MAX) of being in the corresponding state, assuming that both players play optimally from there to the end of the game.
8. Obviously, the minimax value of a terminal state is just its utility.
9. Furthermore, given a choice, MAX will prefer to move to a state of maximum value, whereas MIN prefers a state of minimum value. So we have the following:

$$\text{MINIMAX-VALUE}(n) = \begin{cases} \text{UTILITY}(n) & \text{if n is a terminal state} \\ \max_{s \in Successors(n)} \text{MINIMAX-VALUE}(s) & \text{if } n \text{ is a MAX node} \\ \min_{s \in Successors(n)} \text{MINIMAX-VALUE}(s) & \text{if } n \text{ is a MIN node.} \end{cases}$$

Let us apply these definitions to the game tree in Figure 2.

1. The terminal nodes on the bottom level are already labelled with their utility values.
2. The first MIN node, labelled B, has three successors with values 3, 12, and 8, so its minimax value is 3.
3. Similarly, the other two MIN nodes have minimax value 2.
4. The root node is a MAX node; its successors have minimax values 3, 2, and 2; so it has a minimax value of 3.
5. We can also identify the minimax decision at the root: action a1 is the optimal choice for MAX because it leads to the successor with the highest minimax value.
6. This definition of optimal play for MAX assumes that MIN also plays optimally-it maximizes the worst-case outcome for MAX.

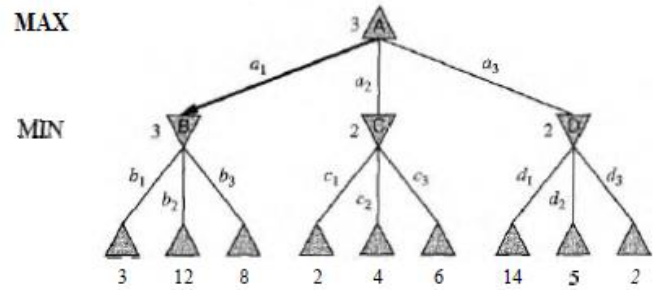


Figure 2: A two-ply game tree. The Δ nodes are "MAX nodes," in which it is MAX'S turn to move, and the Δ nodes are "MIN nodes." The terminal nodes show the utility values for MAX; the other nodes are labelled with their minimax values. MAX'S best move at the root is a1, because it leads to the successor with the highest minimax value, and MIN'S best reply is b1, because it leads to the successor with the lowest minimax value.

The minimax algorithm

1. The minimax algorithm (Figure 3) computes the minimax decision from the current state.
2. It uses a simple recursive computation of the minimax values of each successor state, directly implementing the defining equations.
3. The recursion proceeds all the way down to the leaves of the tree, and then the minimax values are backed up through the tree as the recursion unwinds.

4. For example, in Figure 2, the algorithm first recurses down to the three bottomleft nodes, and uses the UTILITY function on them to discover that their values are 3, 12, and 8 respectively.
5. Then it takes the minimum of these values, 3, and returns it as the backed-up value of node B.
6. A similar process gives the backed up values of 2 for C and 2 for D.
7. Finally, we take the maximum of 3,2, and 2 to get the backed-up value of 3 for the root node.
8. The minimax algorithm performs a complete depth-first exploration of the game tree.
9. If the maximum depth of the tree is m, and there are b legal moves at each point, then the time complexity of the minimax algorithm is O(b m).
10. The space complexity is O(bm) for an algorithm that generates all successors at once, or O(m) for an algorithm that generates successors one at a time.

```
function MINIMAX-DECISION(state) returns an action
  inputs: state, current state in game

  v ← MAX-VALUE(state)
  return the action in SUCCESSORS(state) with value v
```

```
function MAX-VALUE(state) returns a utility value
  if TERMINAL-TEST(state) then return UTILITY(state)
  v ← −∞
  for a, s in SUCCESSORS(state) do
    v ← MAX(v, MIN-VALUE(s))
  return v
```

```
function MIN-VALUE(state) returns a utility value
  if TERMINAL-TEST(state) then return UTILITY(state)
  v ← ∞
  for a, s in SUCCESSORS(state) do
    v ← MIN(v, MAX-VALUE(s))
  return v
```

Figure 3: An algorithm for calculating minimax decisions. It returns the action corresponding to the best possible move, that is, the move that leads to the outcome with the best utility, under the assumption that the opponent plays to minimize utility. The functions MAX-VALUE and MIN-VALUE go through the whole game tree, all the way to the leaves, to determine the backed-up value of a state.

Optimal decisions in multiplayer games

1. Many popular games allow more than two players. Let us examine how to extend the minimax idea to multiplayer games.

2. This is straightforward from the technical viewpoint, but raises some interesting new conceptual issues.
3. First, we need to replace the single value for each nocle with a vector of values.
4. For example, in a three-player game with players A, B, and C,a vector (vA,v ~vc,) is associated with each node.
5. For terminal states, this vector gives the utility of the state from each player's viewpoint.
6. (In two-player, zero-sum games, the two-element vector can be reduced to a single value because the values are always opposite.)
7. The simplest way to implement this is to have the UTILITY function return a vector of utilities.
8. Now we have to consider nonterminal states. Consider the node marked X in the game tree shown in Figure 4.

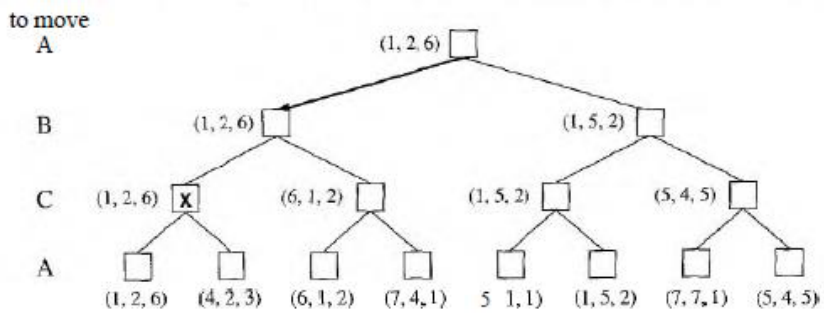


Figure 4: The first three ply of a game tree with three players (A, B, C). Each node is labelled with values from the viewpoint of each player. The best move is marked at the root.

9. In that state, player C chooses what to do. The two choices lead to terminal states with utility vectors (vA = 1, vB = 2, vC = 6) and (vA = 4, vB = 2, vC = 3). Since 6 is bigger than 3, C should choose the first move.
10. This means that if state X is reached, subsequent play will lead to a terminal state with utilities (vA = 1, vB = 2, vC = 6).
11. Hence the backed-up value of X is this vector. In general, the backed-up value of a node n is the utility vector of whichever successor has the highest value for the player choosing at n.
12. Multiplayer games usually involve alliances, whether formal or informal, among the players. Alliances are made and broken as the game proceeds.
13. For example suppose A and B are in weak positions and C is in a stronger position. Then it is often optimal for both A and B to attack C rather than each other, lest C destroy each of them individually.
14. In this way, collaboration emerges from purely selfish behavior. Of course, as soon as C weakens under the joint onslaught, the alliance loses its value, and either A or B could violate the agreement.
15. If the game is not zero-sum, then collaboration can also occur with just two players. Suppose, for example, that there is a terminal state with utilities (vA = 1000, vB = 1000), and that 1000 is the highest possible utility for each player. Then the optimal strategy is for both players to do everything possible to reach this state--that is, the players will automatically cooperate to achieve a mutually desirable goal.

Alpha Beta Pruning

1. The problem with minimax search is that the numbeir of game states it has to examine is exponential in the number of moves.
2. The trick is that it is possible to compute the correct minimax decision without looking at every node in the game tree. That is, we can borrow the

idea of pruning in order to eliminate large parts of the tree from consideration.

3. The particular technique we will examine is called alpha-beta pruning. When applied to a standard minimax tree, it returns the same move as minimax would, but prunes away branches that cannot possibly influence the final decision.

4. Consider again the two-ply game tree from Figure 2. Let's go through the calculation of the optimal decision once more, this time paying careful attention to what we know at each point in the process. The steps are explained in Figure 5. The outcome is that we can identify the minimax decision without ever evaluating two of the leaf nodes.

5. As a simplification of the formula for MINIMAX-VALUE. Let the two unevaluated successors of node C in Figure 5 have values x and y and let z be the minimum of x and y. The value of the root node is given by

6. In other words, the value of the root and hence the minimax decision are independent of the values of the pruned leaves x and y.

$$
\begin{aligned}
\text{MINIMAX-VALUE}(root) &= \max(\min(3,12,8), \min(2,x,y), \min(14,5,2)) \\
&= \max(3, \min(2,x,y), 2) \\
&= \max(3,z,2) \qquad \text{where } z \le 2 \\
&= 3.
\end{aligned}
$$

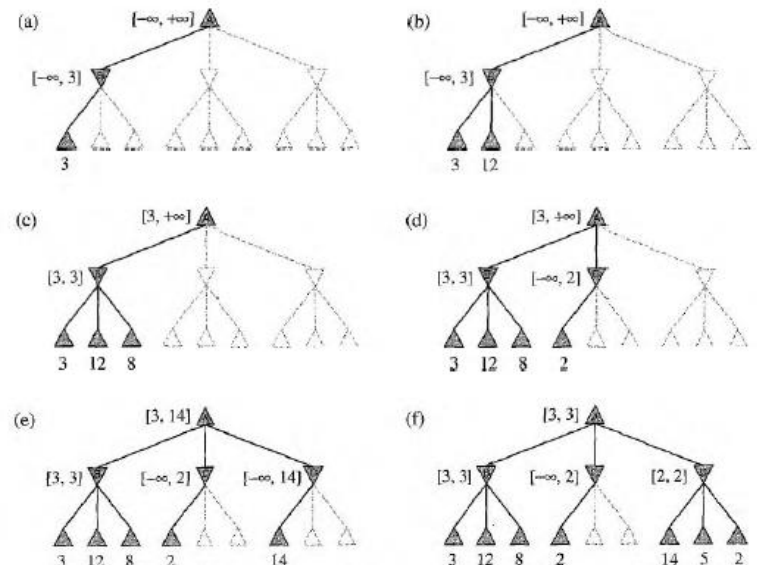


Figure 5: Stages in the calculation of the optimal decision for the game tree in Figure 2.

At each point, we show the range of possible values for each node.

(a) The first leaf below B has the value 3. Hence, B, which is a MIN node, has a value of at most 3.

(b) The second leaf below B has a value of 12; MIN would avoid this move, so the value of B is still at most 3.

(c) The third leaf below B has a value of 8; we have seen all B's successors, so the value of B is exactly 3. Now, we can infer that the value of the root is at least 3, because MAX has a choice worth 3 at the root.

(d) The first leaf below C has the value 2. Hence, C, which is a MIN node, has a value of at most 2. But we know that B is worth 3, so MAX would never choose

C. Therefore, there is no point in loohng at the other successors of C. This is an example of alpha-beta pruning.

(e) The first leaf below D has the value 14, so D is worth at most 14. This is still higher than MAX'S best alternative (i.e., 3), so we need to keep exploring D's successors. Notice also that we now have bounds on all of the successors of the root, so the root's value is also at most 14.

(f) The second successor of D is worth 5, so again we need to keep exploring. The third successor is worth 2, so now D is worth exactly 2. MAX'S decision at the root is to move to B, giving a value of 3.

1. Alpha-beta pruning can be applied to trees of any depth, and it is often possible to prune entire subtrees rather than just leaves.
2. The general principle is this: consider a node n somewhere in the tree (see Figure 6), such that Player has a choice of moving to that node.
3. If Player has a better choice m either at the parent node of n or at any choice point further up, then n will never be reached in actual play. So once we have found out enough about n (by examining some of its descendants) to reach this conclusion, we can prune it.
4. Remember that minimax search is depth-first, so at any one time we just have to consider the nodes along a single path in the tree. Alpha-beta pruning gets its name from the following two parameters that describe bounds on the backed.-up values that appear anywhere along the path:

α = the value of the best (i.e., highest-value) choice we have found so far at any choice point along the path for MAX.

β = the value of the best (i.e., lowest-value) choice we have found so far at any choice point along the path for MIN.

5. Alpha-beta search updates the values of α and β as it goes along and prunes the remaining branches at a node (i.e., terminates the recursive call) as soon as the value of the current node is known to be worse than the current α or β value for MAX or MIN, respectively.

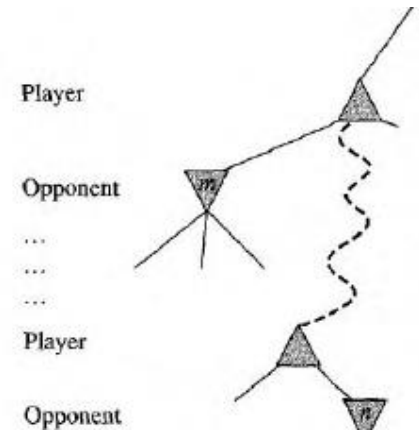


Figure 6: Alpha-beta pruning: the general case. If m is better than n for Player, we will never get to n in play.

1. The complete algorithm is given in Figure 7.
2. The effectiveness of alpha-beta pruning is highly dependent on the order in which the successors are examined.
3. For example, in Figure 5(e) and (f), we could not prune any successors of D at all because the worst successors (from the point of view of MIN) were generated first.
4. If the third successor had been generated first, we would have been able to prune the other two. This suggests that it might be worthwhile to try to examine first the successors that are likely to be best.
5. If we assume that this can be done then it turns out that alpha-beta needs to examine only 0(bd/2) nodes to pick the best move, instead of O(bd) for minimax.

6. In games, repeated states occur frequently because of transpositions- different permutations of the move sequence that end up in the same position.

CONSTRAINT SATISFACTION PROBLEMS (CSP)

Constraint satisfaction problems (CSP) are mathematical problems where one must find states or objects that satisfy a number of constraints or criteria.

A constraint is a restriction of the feasible solutions in an optimization problem.

Some examples for CSP's are:

1. 8-queens problem
2. A crossword puzzle
3. A map coloring problem
4. The Boolean satisfiability problem
5. A crypt arithmetic problem

All these examples and other real life problems like time table scheduling, transport scheduling, floor planning etc. are instances of the same pattern,

A Constraint Satisfaction Problem (or CSP) is defined by a set of variables {X1,X2,....Xn,} and a set of constraints {C1,C2,...,Cm}. Each variable Xi has a nonempty domain D, of possible values.

Each constraint Ci involves some subset of variables and specifies the allowable combinations of values for that subset.

A State of the problem is defined by an assignment of values to some or all of the variables,{Xi = vi, Xj = vj,...}.

An assignment that does not violate any constraints is called a consistent or legal assignment.

A complete assignment is one in which every variable is mentioned, and a solution to a CSP is a complete assignment that satisfies all the constraints. Some CSPs also require a solution that maximizes an objective function.

Example for Constraint Satisfaction Problem :

The map coloring problem. The task of coloring each region red, green or blue in such a way that no neighbouring regions have the same color.

Variables WA, NT, Q, NSW, V, SA, T
Domains $D_i = \{red, green, blue\}$
Constraints: adjacent regions must have different colors
e.g., $WA \neq NT$ (if the language allows this), or
$(WA, NT) \in \{(red, green), (red, blue), (green, red), (green, blue), \ldots\}$

Map of Australia showing each of its states and territories

We are given the task of coloring each region red, green, or blue in such a way that the neighbouring regions must not have the same color.

To formulate this as CSP, we define the variable to be the regions: WA, NT, Q, NSW, V, SA, and T.

The domain of each variable is the set {red, green, blue}.

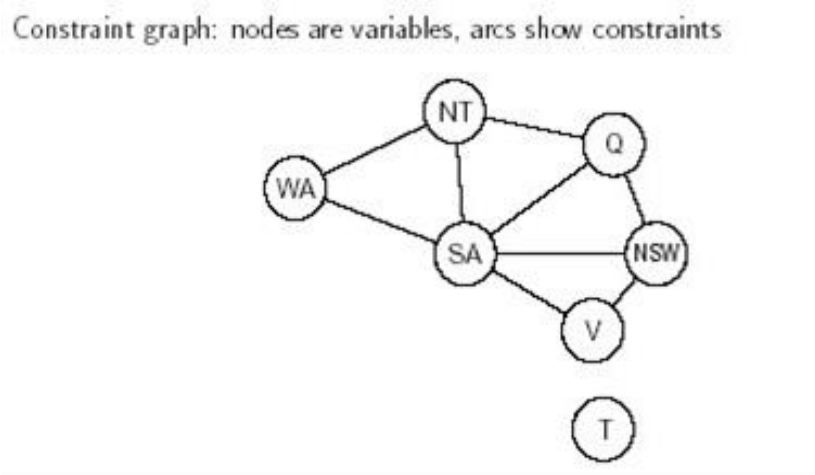
The constraints require neighbouring regions to have distinct colors: for example, the allowable combinations for WA and NT are the pairs{(red,green),(red,blue),(green,red),(green,blue),(blue,red),(blue,green)}. (The constraint can also be represented as the inequality WA ≠ NT)

There are many possible solutions, such as

{WA = red, NT = green, Q = red, NSW = green, V = red, SA = blue,T = red}.

Constraint Graph: A CSP is usually represented as an undirected graph, called constraint graph where the nodes are the variables and the edges are the binary constraints.

The map-coloring problem represented as a constraint graph.




```
function BACKTRACKING-SEARCH(csp) returns solution/failure
  return RECURSIVE-BACKTRACKING({ }, csp)

function RECURSIVE-BACKTRACKING(assignment, csp) returns soln/failure
  if assignment is complete then return assignment
  var ← SELECT-UNASSIGNED-VARIABLE(VARIABLES[csp], assignment, csp)
  for each value in ORDER-DOMAIN-VALUES(var, assignment, csp) do
     if value is consistent with assignment given CONSTRAINTS[csp] then
        add {var = value} to assignment
        result ← RECURSIVE-BACKTRACKING(assignment, csp)
        if result ≠ failure then return result
        remove {var = value} from assignment
  return failure
```

CSP can be viewed as a standard search problem as follows:

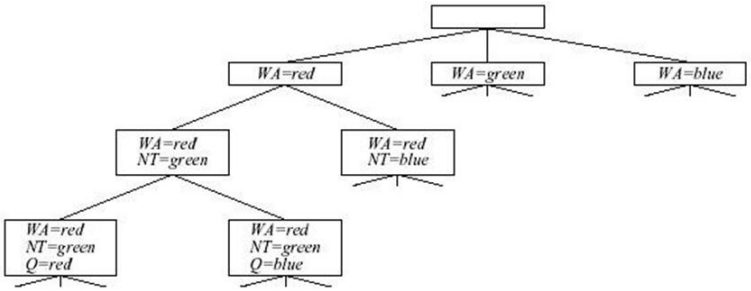
- **Initial state**: the empty assignment {}, in which all variables are unassigned.
- **Successor function**: a value can be assigned to any unassigned variable provided that it does not conflict with previously assigned variables.
- **Goal test**: the current assignment is complete.
- **Path cost**: a constant cost (E.g., 1) for every step.

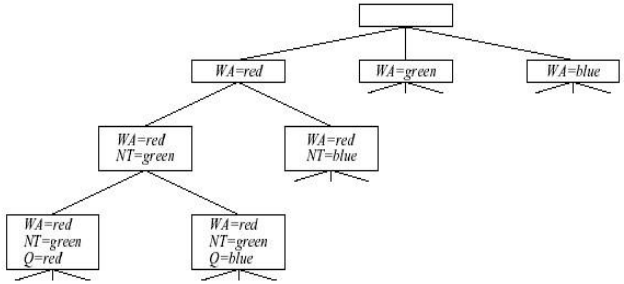
  Every solution must be a complete assignment and therefore appears at depth n if there are n variables. So Depth first search algorithms are popular for CSPs.

Backtracking Search for CSPs

The term backtracking search is used for depth-first search that chooses values for one variable at a time and backtracks when a variable has no legal values left to assign.

Part of search tree generated by simple backtracking for the map coloring problem





Forward checking

- One way to make better use of constraints during search is called forward checking.
- Wheneve ra variable X is assigned,the forward checking process looks at each unassigned variableY that is connected to X by a constraint and deletes from Y 's domain any value that is inconsistent with the value chosen forX.

	WA	NT	Q	NSW	V	SA	T
Initial domains	R G B	R G B	R G B	R G B	R G B	R G B	R G B
After *WA=red*	R	G B	R G B	R G B	R G B	G B	R G B
After *Q=green*	R	B	G	R B	R G B	B	R G B
After *V=blue*	R	B	G	R	B		R G B

Figure 5.6 The progress of a map-coloring search with forward checking. $WA = red$ is assigned first; then forward checking deletes *red* from the domains of the neighboring variables NT and SA. After $Q = green$, *green* is deleted from the domains of NT, SA, and NSW. After $V = blue$, *blue* is deleted from the domains of NSW and SA, leaving SA with no legal values.

Constraint propagation

- Although forward checking detects many inconsistencies, it does not detect all of them.
- Constraint propagation is the general term for propagating the implications of a constraint on one variable on to other variables.

Arc Consistency

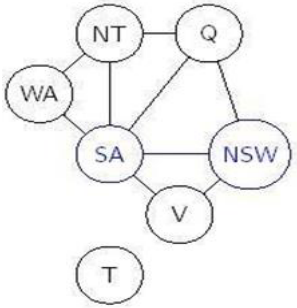


Figure: Australian Territories

- One method of constraint propagation is to enforce **arc consistency**
  - Stronger than forward checking
  - Fast
- *Arc* refers to a *directed* arc in the constraint graph
- Consider two nodes in the constraint graph (e.g., *SA* and *NSW*)
  - An arc is **consistent** if
  - For every value x of *SA*
  - There is some value y of *NSW* that is consistent with x
- Examine arcs for consistency in *both* directions

Local Search for CSPs

- Local search algorithms good for many CSPs
- Use complete-state formulation
  - Value assigned to every variable
  - Successor function changes one value at a time
- Have already seen this
  - Hill climbing for 8-queens problem (AIMA § 4.3)
- Choose values using **min-conflicts** heuristic
  - Value that results in the minimum number of conflicts with other variables